How Execution Features Relate to Failures

An Empirical Study and Diagnosis Approach

MARIUS SMYTZEK, CISPA Helmholtz Center for Information Security, Germany
MARTIN EBERLEIN, Humboldt-Universität zu Berlin, Germany
LARS GRUNSKE, Humboldt-Universität zu Berlin, Germany
ANDREAS ZELLER, CISPA Helmholtz Center for Information Security, Germany

Fault localization aims to identify code regions responsible for failures. Traditional techniques primarily correlate statement execution with failures; however, program behavior involves diverse execution features, including variable values, branch conditions, and definition-use pairs, which can provide richer diagnostic insights.

This paper comprehensively investigates execution features for fault understanding, addressing two complementary goals. First, we conduct an empirical study of 310 bugs across 20 projects, analyzing 17 execution features and assessing their correlation with failure outcomes. Our findings suggest that fault localization benefits from a broader range of execution features: (1) *Scalar pairs* exhibit the strongest correlation with failures; (2) Beyond line executions, *def-use pairs* and *functions executed* are key indicators for fault localization; and (3) Combining *multiple features* enhances effectiveness compared to relying on individual features.

Second, building on these insights, we introduce a debugging approach that learns relevant features from labeled test outcomes. The approach extracts fine-grained execution features and trains a decision tree to differentiate passing and failing runs. The trained model generates fault diagnoses that explain the underlying causes of failures.

Our evaluation demonstrates that the generated diagnoses achieve high predictive accuracy. These interpretable diagnoses empower developers to debug software efficiently by providing deeper insights into failures.

CCS Concepts: • **Software and its engineering → Software testing and debugging**; *Software maintenance tools*; • **General and reference → Empirical studies**; **Metrics**; **Measurement**; • **Theory of computation → Oracles and decision trees**; *Active learning*.

Additional Key Words and Phrases: fault localization, statistical debugging, execution features, empirical study, automated debugging

1 Introduction

Debugging is the process of identifying and fixing faults in a program. Whether automated or manual, a central task in debugging is *fault localization*, i.e., identifying the locations in the code that are likely to contain the root cause of a failure. Fault localization techniques typically rely on analyzing program executions to identify the locations most relevant to the failure.

Authors' Contact Information: Marius Smytzek, CISPA Helmholtz Center for Information Security, Saarbrücken, Germany, marius.smytzek@cispa.de; Martin Eberlein, Humboldt-Universität zu Berlin, Berlin, Germany, martin.eberlein@hu-berlin.de; Lars Grunske, Humboldt-Universität zu Berlin, Berlin, Germany, grunske@hu-berlin.de; Andreas Zeller, CISPA Helmholtz Center for Information Security, Saarbrücken, Germany, zeller@cispa.de.

ACM 1557-7392/2025/12-ART
https://doi.org/10.1145/3783989

As a classic example, consider Figure 1, used by its authors to demonstrate the TARANTULA localization [20]. The `middle()` function returns neither the minimum nor the maximum of its arguments `x`, `y`, and `z`. However, `middle()` is faulty, as `middle(2, 1, 3)` returns 1 rather than 2.

TARANTULA runs a suite of tests on the program, correlating the failures with the execution of code lines. Given the four test cases from [20], Line 6 is executed only in the one failing test, which strongly correlates with the failure. (Line 6 also *is* the faulty line.) However, the effectiveness of line-based fault localization depends heavily on the test cases: Adding a test `middle(1, 1, 3)` breaks the strong correlation; Line 6 is executed, but the test returns 1, passing.

The advantage of line coverage is that it is universal; pretty much any programming language provides a means to measure it. The coverage of a line, however, is just one of many features of a program execution. Besides line coverage, other features could also correlate with failures, such as variable values, definition-use pairs (i.e., pairs of locations where values are defined and later used), branch conditions, and more.

Moreover, debugging the locations alone may not be sufficient to understand the root cause of a failure from a developer's perspective. A developer relies on questions that can not be answered by a single line of code, such as: (1) Why did the failure occur? (2) What led to the failure? (3) What contributed to the failure?

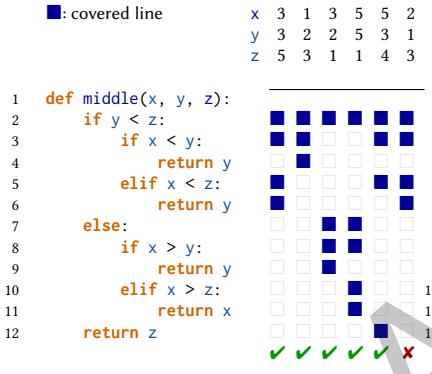


Fig. 1. Statistical fault localization [21]. The `middle()` function takes three values and returns the one that is neither the smallest nor the largest; execution of Line 6 correlates most closely with the failure.

Recent toolkits allow retrieving all of these features from a program's execution, thus providing a much more ideal choice of execution features to use for fault localization. The SFLKIT [50] workbench for Python programs, among many other execution features, allows extracting *scalar pairs* that occurred during execution, i.e., relationships between two variables. Aided by SFLKIT on the `middle()` example, we could extract a feature that the test fails if `y < x` holds. However, this feature is only one of many that could be extracted from the execution.

Do various execution features work better than lines to characterize failures or locate faults? Moreover, which execution features would these be? To answer these questions, we conduct an empirical study to determine *which execution features are most relevant to failures*. During program execution, we collect 17 different execution features inspired by test coverage criteria, ranging from branches covered to definition-use pairs and many more, and assess how effective these features are in detecting and localizing faults. Our study encompasses 310 faults from 20 open-source projects from the TESTS4PY dataset [49], which is based on the faults in BUGSINPY [58].

To the best of our knowledge, this is the first study to analyze and compare the interplay of a large set of execution features with software failures. Our findings suggest that fault localization *should rely on a larger and more diverse range of execution features:*

(1) Features that capture data *and* control flow, particularly string properties and scalar pairs, showed the highest correlation with failures;
(2) Besides lines executed, *def-use pairs* and *functions executed* are the best features for localizing faults; and
(3) Considering *several features* yields better fault localization than executed lines alone.

Insights such as these can (1) help in identifying root causes of program failures more accurately, (2) provide better hints for automated code repairs, and (3) open up new possibilities for debugging techniques and research.

These empirical findings demonstrate that diverse execution features provide valuable insights into fault understanding. However, to demonstrate their practical utility, we must show how these insights can be translated into actionable debugging tools. Moreover, we introduce a debugging approach that learns relevant execution features and generates a diagnosis for a faulty program, explaining the underlying causes of the failure. We leverage the execution features, which we have shown provide valuable insights, are effective in fault localization, and strongly correlate with failures to train a *classifier* for predicting failures based on the execution features we collected. In the case of `middle()` and the executions in Figure 1, for instance, the model determined by our execution-feature-driven-debugging (or short EFDD) approach predicts a failure if two properties hold: (1) The `return y` in Line 6 is executed, and (2) y is less than x. These two conditions form an accurate *diagnosis*—the failure occurs precisely under these conditions that a developer can understand and act upon, i.e., the fix needs to be in Line 6, which returns y, but y is less than x, and from the previous Line 5 we know that x is less than z, hence x should be returned.

This example illustrates how EFDD transcends traditional spectrum-based fault localization by providing *actionable diagnostic insight* rather than mere suspiciousness rankings. While TARANTULA would identify Line 6 as suspicious based on execution correlation, EFDD explains *when* and *why* Line 6 causes failures: specifically when returning y while $y < x$. This diagnosis directly suggests the nature of the algorithmic error and points toward the appropriate fix, demonstrating how execution feature analysis can provide causal understanding beyond statistical correlation alone.

Note that our debugging approach does not require an explicit formal specification of correct behavior (which, even for a seemingly simple function like `middle()`, can be complex to specify formally). While our approach does rely on a test suite, which can be considered a form of behavioral specification, it works with any existing tests and requires no additional specification effort from developers. Nor does it assume a *generic* failure such as a crash (for which the execution feature that most correlates with failure is obvious.[1]). Moreover, our approach is not limited to specific kinds of tests, such as unit tests, but works with any test suite that provides labeled outcomes, including inputs for which we know the correct and incorrect behavior.

To the best of our knowledge, this is the first approach that learns relevant features based on the execution and generates fault diagnoses that explain the underlying causes of failures.

Overall, we make the following contributions:

An *empirical study* assessing how execution features relate to failures. We conduct a comprehensive *empirical study* investigating the effectiveness of 17 execution features in detecting and localizing faults across 310 bugs from 20 projects, establishing which features are most relevant to failures.

An investigation on *combinations of features.* We systematically investigate how collecting and leveraging *multiple features* from program executions can improve fault localization beyond traditional line-based approaches.

A practical *debugging approach.* We demonstrate how empirical insights about execution features can be translated into practical debugging tools by presenting EFDD, which generates interpretable fault diagnoses from execution features.

A comprehensive *evaluation.* We evaluate the empirical effectiveness of execution features and the practical utility of feature-driven diagnoses, showing that diverse execution features provide actionable insights for developers.

A comprehensive *dataset and tooling.* We provide a dataset of faults and executions, including all events associated with corresponding executions, along with open-source tools to facilitate further research in feature-driven debugging.

[1]It is the crash.

The remainder of this paper is organized as follows. Section 2 describes the execution features analyzed in our study. Additionally, it introduces the concept of using multiple execution features for fault localization and diagnosis in Section 2.5. Section 3 outlines the design of our study, which evaluates the effectiveness of these features in fault detection and localization. This section is concluded by Section 3.6, which presents the results and implications of our study. After introducing execution features and their value for debugging, we present a debugging approach that leverages these features to generate fault diagnoses in Section 4. Section 5 evaluates the effectiveness of our debugging approach in generating fault diagnoses. We address potential threats to validity in Section 6 and review related work in Section 7. Finally, Section 8 concludes the paper and suggests avenues for future research. All data and tools developed for this study are available as open source (Section 8).

2 Execution Features

Our study analyzes program executions to identify features most pertinent to failures, aiming to capture behavioral nuances of the program during execution. We collect diverse data points, such as executed lines, variable states, method calls, and condition outcomes, recorded as *events* across different execution instances. These events encapsulate control and data flow, providing insight into the program's operational context.

From these events, we derive *features* that represent whether specific behaviors occurred during execution or if conditions relevant to failure were met. By statistically analyzing these features, we identify those most strongly associated with failure, extending beyond the typical scope of statement coverage used in traditional fault localization.

2.1 Statistical Fault Localization

Statistical fault localization techniques aim to pinpoint code locations most associated with failures by correlating program execution data with the occurrence of failures. Many of these techniques rely on *spectra*-based information, typically in the form of line or block coverage, to identify suspect areas. For instance, the TARANTULA tool [20] calculates the *suspiciousness* of each line based on its coverage across failing and passing test cases, with lines more frequently executed by failing tests flagged as potential fault sites. Beyond line coverage, some techniques expand to other execution details. Liblit et al. [34] focus on *branches*, identifying branches that appear disproportionately in failing test cases. Similarly, Santelices et al. [48] introduce *definition-use pairs* as a data flow feature for fault localization, highlighting variable definitions and uses that correlate with failure. Additional work on def-use pairs includes Masri's information flow coverage approach [38], which models complex interactions between program elements through data flow relationships, as well as empirical studies [39] that identify factors affecting the effectiveness of def-use-based fault localization. Although these methods improve diagnostic granularity, they typically lack direct mappings to specific fault locations in the code, limiting their practical diagnostic utility. Traditional methods also rely on specific metrics, such as TARANTULA [20], OCHIAI [2], and JACCARD [6], which calculate line suspiciousness by comparing the presence of features across failing and passing executions. While these metrics have shown utility, they primarily operate on executed lines or specific statements, potentially overlooking richer data sources within the program execution.

In our study, we expand statistical fault localization by applying suspiciousness metrics to a diverse set of *execution features* beyond line and branch coverage alone. By integrating features such as data flow events, variable values, and condition outcomes, we aim to capture a more comprehensive view of program behavior that can more accurately reflect fault locations. These broader investigation targets of our study enable us to assess whether a feature-centric perspective, rather than a line-centric one, can yield improved fault localization outcomes.

2.2 Events

During execution, we capture events that reflect the program's behavior. These events are categorized using standard test coverage metrics and further enriched to support broader fault analysis. This collection process involves instrumenting a faulty program, allowing us to obtain a detailed execution trace during any program run. The primary types of captured events are outlined below:

Lines. We monitor the execution of each program line, recording every executed line as an event in the trace. This fine-grained coverage information tracks statement-level execution, providing the most granular view of program behavior. Line events form the foundation for traditional spectrum-based fault localization, helping pinpoint exact fault locations within the source code. Each line event includes the line number and source file, enabling precise mapping between execution traces and source code locations.

Branches. Whenever a branch is executed, it is logged in the trace. A branch event captures *which execution path* was taken (e.g., the `then` or `else` branch of an `if` statement), corresponding to a specific control flow edge in the program's control flow graph. Branches can originate from various control structures, including conditional statements (`if/else`), exception handling (`try/except/finally`), or pattern matching (`match/case`). Each branch event records the source location and a unique branch identifier, providing insights into the program's decision-making process and helping detect logical errors in conditional logic and exception handling paths.

Functions. All function calls, including their names, are logged. Function events capture the program's modular execution flow, recording function entry and exit points, parameter values, and call stack information. These events also capture method calls on objects and recursive calls. Each function event stores information about the executed function, e.g., the function name and the source location.

Variable Definitions. An event is generated whenever a variable is defined, capturing its name, value, and type, including initial assignments, reassignments, parameter bindings, loop variable assignments, and context manager variables (e.g., bindings in `with` statements). Variable definition events track the program's state evolution at key mutation points, recording the variable name, value, and source location. This comprehensive tracking helps identify issues related to variable initialization, type inconsistencies, and unintended changes.

Variable Uses. Events are recorded when variables are referenced or accessed, capturing read operations that complement the definition events, including variable lookups in expressions, function arguments, return statements, and attribute access on objects. Each use event records the variable name and the source of usage. Variable use events enable def-use pair analysis, helping identify issues with incorrectly initialized variables, wrong variable references, scope resolution problems, and data flow anomalies that may indicate logical errors.

Return Values. Function return values are logged to enhance the trace, capturing both explicit return statements and implicit returns (e.g., functions that end without an explicit return). Each return event records the returned value, its type, and the context in which the returning function was invoked. This tracking enables a deeper understanding of function outcomes and their influence on program execution, particularly for diagnosing faults related to incorrect return values, missing return statements, or functions returning unexpected types. Return value tracking is crucial for analyzing data flow through function boundaries and identifying discrepancies between expected and actual outputs of functions.

Loops. We track comprehensive loop execution information, capturing all entry and exit events for loop types, as well as iteration counts. Loop events record the location of the loop and the information about its state (i.e., entry, exit, and iteration). This detailed loop tracking helps identify incorrect loop iterations, off-by-one errors, incorrect loop termination conditions, performance issues related to excessive iterations, and logical errors in loop variable manipulation.

Conditions. The evaluation results of all conditional expressions, including nested and compound conditions, are recorded with their Boolean outcomes. Unlike branches, which capture *which path* was taken, conditions capture *how each conditional expression evaluated* (i.e., whether it resulted in `true` or `false`), including simple

comparisons, compound boolean expressions with logical operators (`and`, `or`, `not`), and other expressions that can be interpreted as boolean conditions within the context of control flow. Each condition event records the expression, evaluation result, and the source location. Condition events provide insights into the program's decision-making process, helping identify issues related to incorrect condition evaluations, logical errors, and unintended control flow.

This comprehensive instrumentation ensures that each program execution, whether from a unit test or an arbitrary input, generates a detailed trace of these defined events. After instrumentation, we iteratively execute the provided test cases to assemble an event trace for each test case. Given that test cases include labels indicating their "pass" or "fail" status, we can differentiate between passing and failing traces.

By capturing a wide range of events, we gain a holistic view of the program's execution, which enhances fault localization and diagnosis. This detailed event trace forms the foundation of our subsequent analysis, particularly in our debugging approach, where we leverage machine learning to identify patterns and correlations indicative of faults. Understanding the exact sequence of events leading to a fault enables developers to pinpoint root causes and implement appropriate fixes efficiently.

2.3 Features

We derive *features* from the event traces that capture meaningful execution behaviors. These features indicate the program's behavior under both normal and failing conditions. Inspired by statistical fault localization research [19, 20, 34, 48], we construct a total of 17 features, which can be categorized into four key groups: coverage information, value properties, condition outcomes, and triggered exceptions. An overview of the feature structure is provided in Table 1.

Coverage. We focus on line execution, as suggested in spectrum-based fault localization [20], and also consider specific branches taken during execution [34]. Additionally, we capture definition-use pairs as proposed by Santelices et al. [48], which have been further studied for their effectiveness in various contexts [25, 38, 39]. Loop coverage is assessed by tracking whether a loop was skipped, entered once, or executed multiple times.

Values. We extract features based on logged *values* in events, such as variable assignments and function return values. Examples include detecting whether a number is zero, a variable is `NULL`, or a return value contains non-ASCII characters. We also compare values to other variables to create *scalar pairs*, an approach influenced by Liblit et al. [34]. Furthermore, for variables with lengths, such as strings or lists, we classify their length as zero, one, or more than one.

Our approach focuses on directly tracking basic scalar types (integers, floats, strings, booleans) rather than complex objects. When complex objects are encountered, we do not capture the entire object structure; instead, we track individual field accesses as they occur. For instance, if an object `person` has properties `name` and `age`, we may record events for `person.name` and `person.age` as separate variable uses when these properties are accessed. Scalar pairs are created between these individual field values and other variables in scope, enabling fine-grained analysis of object property relationships. This approach allows us to maintain the interpretability of our decision trees while capturing relevant object-level information through property-specific features.

Conditions. Since *branch conditions* are already recorded in our events, they are directly incorporated as features.

Exceptions. *Exceptions* play a critical role in program control flow and can hint at fault occurrences. Hence, we include a feature that records whether a function exited normally or with an exception.

A feature is considered *satisfied* for a specific execution if the related event was triggered and the feature's defined condition was met during execution. For instance, a feature that monitors the execution of a particular line is satisfied if that line was executed during the program run. Similarly, a feature based on a variable's value is satisfied if the variable was defined during execution and met a specified condition, such as an integer value less

Table 1. Feature Structure Overview. Feature types, their possible values, and measurement entities used in our execution feature analysis. Binary features can be satisfied (1) or not (0). Tertiary features can additionally be unobserved (-1). The measurements column shows specific program elements or conditions that each feature type tracks during execution.

Feature Type	Representation	Possible Values	Measurements & Examples
Coverage Features			
Lines	Binary	{0, 1}	Individual line numbers (e.g., Line 6, Line 12)
Branches	Tertiary	{-1, 0, 1}	Branch edges (e.g., `if-then`, `if-else`, `try-except`)
Functions	Binary	{0, 1}	Function calls (e.g., `middle()`)
Def-Use Pairs	Binary	{0, 1}	Variable definition-use relationships (e.g., `DefUse(x,1,3)`)
Loops	Binary	{0, 1}	Separate features: not executed, executed once, executed multiple times
Value-Based Features			
Scalar Pairs	Tertiary	{-1, 0, 1}	Variable comparisons (e.g., `x < y`)
Variable Values	Tertiary	{-1, 0, 1}	Value properties (e.g., `x == 0`, `y is None`)
Return Values	Tertiary	{-1, 0, 1}	Function return properties (e.g., returns `None`, returns `< 0`)
String Properties	Tertiary	{-1, 0, 1}	String characteristics (e.g., contains digits, special chars)
Length Properties	Binary	{0, 1}	Separate features: empty, single element, multiple elements
Control Flow Features			
Conditions	Tertiary	{-1, 0, 1}	Conditional expression evaluations (e.g., `x < y`, `z in list`, `x and y`)
Exceptions	Binary	{0, 1}	Exception occurrence (exception raised)

than zero. Once derived from events, these features are analyzed using statistical fault localization techniques to assess their relevance to failures. We identify features most strongly associated with faults by correlating feature satisfaction across individual test cases with the occurrence of failures. This correlation helps pinpoint execution characteristics that indicate likely fault locations.

Our feature selection is inspired by the extensive set of events, spectra, and predicates available in SFLKit [50], providing a robust foundation for constructing and evaluating feature conditions for fault localization.

As an example, consider the `middle()` function from Figure 1. Figure 2 illustrates the features derived from executing this function. We collect executed lines, branches, definition-use pairs, scalar pairs, and conditions, rather than all possible features, to maintain conciseness. For instance, a scalar pair feature is collected when a variable is defined and constructed by comparing the variable to another variable, e.g., `y < x`, for all compatible types. This feature holds if the comparison evaluates to `True` and does not hold if it evaluates to `False`. If a feature is not recorded for an execution, we assign a value indicating its absence. Similarly, a definition-use pair feature is collected when a variable is used, incorporating both its definition and usage, e.g., `y` defined in line 1 and used in line 3. More straightforward features, such as line execution, branch outcomes, or condition evaluations, are collected whenever the corresponding event occurs, e.g., the execution of line 2 or the evaluation of `y < z`. Again, absent features are assigned values indicating their non-occurrence.

Later in our debugging approach, we transform these features into binary and tertiary representations. A binary feature either holds (1) or does not hold (0) during a run, while a tertiary feature can hold (1), not hold (0), or be unobserved (−1). For example, line execution is a binary feature, as it can only be executed or not executed. In contrast, condition evaluation is tertiary, as it can evaluate to `true`, `false`, or remain unobserved.

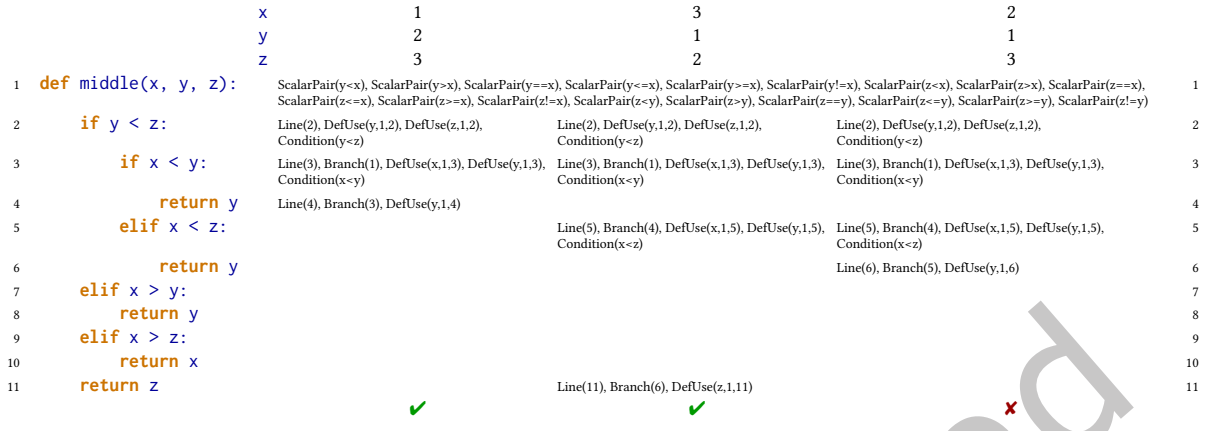


Fig. 2. Collecting Features. For the `middle()` function, we derive features from the execution. For example, we show the collection of executed lines, branches, definition-use pairs, scalar pairs, and conditions. We show when a feature is collected by assigning it to the corresponding line in the code. The values in parentheses indicate the parameters of a feature; for lines, the line number; for branches, the branch ID; for definition-use pairs, the variable and the lines for the definition and use; for scalar pairs, the two variables, and the applied comparison; for conditions, the actual condition.

2.3.1 Handling Multiple Feature Values. When a code snippet is executed multiple times during a single program run, a feature may take on multiple values. For example, a branch condition might evaluate to both `true` and `false` during different iterations of a loop, or a scalar pair comparison might yield different results when variables change values throughout execution. To handle such cases, we apply the following aggregation strategy: (1) For **binary features** (e.g., line execution), the feature is satisfied if the event occurs *at least once* during execution. (2) For **tertiary features** (e.g., condition evaluation), we prioritize observed values over unobserved ones. If the feature is recorded to be both `true` and `false`, the feature will be set to `true` (1). We counteract the potential bias of multiple evaluations by additionally tracking the inverted feature, i.e., the feature is set to `true` (1) if the feature is recorded as `false` at least once. This approach ensures that our feature vectors capture the essential execution characteristics while maintaining consistency across different execution patterns.

Since not all runs include every feature, we assign default values during feature vector construction to normalize the dataset. The default value for binary features is `false`, while for features with three states, it is `not observed`. This standardization ensures that feature vectors maintain consistent size and structure across all runs. Note that when it comes to statistical fault localization, the distinction between `not observed` and `false` is not crucial, as both indicate the absence of the feature and do not contribute to the correlation with failures. For calculating suspiciousness scores using standard coefficients like TARANTULA or OCHIAI, we treat both `not observed` (−1) and `false` (0) as indicating that the feature condition was not met. Only features with value `true` (1) are considered as evidence supporting the correlation between the feature and failure occurrence. This approach simplifies the statistical analysis while maintaining the interpretability of results, as the key insight lies in identifying which execution behaviors are present during failures, rather than distinguishing between different types of absence.

2.3.2 Concrete Example of Tertiary Feature Calculation. Consider a condition feature `x < y` evaluated across four test executions. (1) Test 1 (PASS), where condition evaluates to `true` (1); (2) Test 2 (PASS), where condition not encountered (−1); (3) Test 3 (FAIL), where condition evaluates to `false` (0); and (4) Test 4 (FAIL), where condition evaluates to `true` (1). For suspiciousness calculation using TARANTULA, we count the feature as satisfied when

the value equals 1, yielding: passed tests with feature = 1, failed tests with feature = 1, total passed = 2, total failed = 2. The TARANTULA suspiciousness score becomes $\frac{1/2}{1/2+1/2} = \frac{0.5}{1} = 0.5$, indicating moderate correlation with failures.

2.3.3 Def-Use Pairs vs. Line Coverage: A Concrete Example. To illustrate how def-use pairs can outperform simpler coverage features like line execution, consider a modified version of the `middle()` function in Figure 3 that uses an intermediate variable to store the result.

In this version, there is a fault at line 7 where `m` is incorrectly assigned `y` instead of `x`. For a failing test case like `middle(2, 1, 3)`, traditional line coverage analysis would show that lines 2, 3, 6, 7, and 13 are executed in both passing and failing runs, providing limited discriminative power for fault localization.

However, def-use pair analysis reveals the crucial difference: the def-use pair `DefUse(m, 7, 13)` captures the flow from the faulty assignment at line 7 to the return statement at line 13. This def-use pair is satisfied in failing executions, making it a highly discriminative feature for identifying the fault location. In contrast, passing executions exhibit different def-use pairs such as `DefUse(m, 2, 13)` or `DefUse(m, 5, 13)`, depending on the execution path.

```
1  def middle(x, y, z):
2      m = z
3      if y < z:
4          if x < y:
5              m = y
6          elif x < z:
7              m = y
8      elif x > y:
9          m = y
10     elif x > z:
11         m = x
12     return m
```

Fig. 3. Modified `middle()` Function. The function now uses an intermediate variable `m` to store the incorrectly assigned result in line 7.

This example demonstrates why def-use pairs consistently achieve superior fault localization performance in our empirical study—they capture the essential data flow relationships that directly relate to fault manifestation. At the same time, simple line coverage may miss these critical connections between variable definitions and their problematic uses.

2.4 Soundness and Completeness of Features

2.4.1 Soundness. Our feature selection is designed to be sufficiently detailed to accurately represent executions. By ensuring that each set of features is unique to its corresponding execution or closely similar, we avoid incorrect outcomes, thereby affirming the soundness of our features in distinguishing between passing and failing runs. Soundness is crucial because it guarantees that the features we select are reliable indicators of the execution's behavior. This reliability is essential for debugging and optimizing the system. It ensures that the features accurately represent the underlying processes, reducing false positives and negatives and leading to more accurate insights.

2.4.2 Completeness. While we strive for completeness by covering a broad range of general scenarios, there are instances where our features may fall short of expectations. For example, consider a fault arising from a function that should have been executed. Our current features might not detect such faults because they focus on observable effects rather than potential omissions. Although we may pick up indirect signs of this fault, these indicators are not always definitive, but may be sufficient to identify the fault. However, we cannot guarantee the latter scenario. Enhancing our feature set to capture such scenarios is possible, but some unique situations will inevitably still challenge the completeness of our feature coverage. Completeness is vital because it ensures that all relevant aspects of the execution are considered, providing a holistic view of the system's behavior. This comprehensive coverage is necessary to identify and address all potential issues, improving the system's robustness and reliability. To achieve greater completeness, we can incorporate additional features that capture a more comprehensive range of execution behaviors, including those that are less common or more subtle. However,

it is essential to balance the need for completeness with the practical constraints of feature selection, such as computational overhead and data availability.

2.5 Multi-Feature Fault Localization

While each feature could be leveraged to localize faults, combining these features that highlight their benefits while mitigating their drawbacks could provide a more comprehensive fault localization. Hence, we propose to leverage *multi-feature techniques*, similar to those used by Santelices et al. [48], which utilize these derived features and combine the strengths of various features, considering the different information sources and their relation to failures, to localize faults more accurately and precisely. These techniques aim to verify whether combining multiple features can provide a more comprehensive fault localization than relying on a single feature alone.

We have implemented such a proof-of-concept that evaluates all features simultaneously, calculating the correlation of each based on a provided coefficient, e.g., TARANTULA, which considers the number of passed and failed test cases that satisfy the feature. We then order the features based on their suspiciousness scores, identifying the most relevant features that contributed to the failure. Each feature corresponds to a specific code location, such as a single line or a block of lines, allowing us to assign a suspiciousness score to each line. A challenge arises when multiple features correspond to the same line, necessitating a method for combining their suspiciousness scores. We provide an interface in our technique to utilize various metrics for integrating the suspiciousness of multiple features that map to the same line. Intuitively, we can select among metrics such as maximum, minimum, median, or average suspiciousness to determine the final score for each line in the program, where we consider mainly the maximum suspiciousness as the metric of choice. After calculating the suspiciousness scores for each line, we can rank them based on their scores, identifying the lines most likely to contain the root cause of the failure, as traditional fault localization techniques do.

3 Empirical Study

3.1 Research Questions

In our empirical study, we aim to investigate the effectiveness of the presented execution features in detecting and localizing faults. This investigation serves two complementary purposes: first, to understand which execution features are most effective for fault analysis, and second, to establish the foundation for building practical debugging tools. Hence, we formulate the following research questions to guide our empirical analysis:

RQ1: **Correlation.** To what degree does each execution feature correlate with the presence of failures?
RQ2: **Localization.** How well does each execution feature localize the faults in the program?
RQ3: **Multi-Features.** Can we combine execution features to provide a more comprehensive fault localization than a single feature?

We want to emphasize that the goal of our evaluation in RQ1 to RQ3 is not to improve the state-of-the-art in fault localization but to assess the effectiveness of execution features in detecting and localizing faults. Our empirical study aims to demonstrate that while line-based features are practical, they are not optimal, and diverse execution features can provide a superior understanding of faults. This foundation enables future research in feature-driven debugging that does not rely on lines or, more broadly, a single feature alone.

In the traditional evaluation of fault localization, faulty lines are leveraged to assess the effectiveness of a technique, which comes from a fixed version of the program that may not be the only possible fix for the fault, potentially leading to bias in the evaluation. To address this issue, we introduced RQ1 that shifts the focus to the correlation of features with faults as an initial assessment of the features' effectiveness in identifying the presence of faults.

3.2 Subjects

Our study utilizes open-source projects from the Tests4Py dataset [49], a curated collection of Python projects with documented faults and corresponding test cases. Each project in this dataset includes a faulty and a fixed version, where the faults have been addressed. To ensure fault validity, we first executed the provided test cases on the faulty versions, confirming that the expected failures occurred and that each fault had at least one passing test case. This passing test case is essential for calculating meaningful correlations in fault localization. We then verified the fixed versions by rerunning the test cases, confirming that all tests passed and the faults were correctly resolved. Through this process, we successfully reproduced failures and validated test case accuracy for 310 subjects across 20 projects in the dataset. These 310 subjects or individual faulty versions were selected for our empirical analysis as they provide a reliable and reproducible basis for evaluating fault localization techniques.

3.3 Collecting the Data

For each project, we collected the trace of events during the execution of the test cases for the faulty version of the program. Tests4Py provides a set of test cases for each project relevant to the failure. This set includes test cases that trigger the failure and those that explore related functionality without triggering the bug, such as executing the same function with different parameters. We ensured that all test case executions were correctly categorized, confirming that the test cases expected to fail did indeed fail while collecting the event traces.

Our feature collection operates on the *entire codebase* during test execution, not just on code segments executed by failing test cases. When a test case runs (whether passing or failing), we collect execution features from all code paths traversed during that specific test execution. This comprehensive approach ensures that we capture: (1) Features from code executed by both passing and failing test cases, enabling comparative analysis; (2) Complete execution context, including library calls and helper functions that may be relevant to fault diagnosis; (3) Comprehensive data flow information across the entire program structure; and (4) Features from code paths that may be indirectly related to faults but not immediately obvious from failing test case execution alone. Our whole-program instrumentation is essential for statistical fault localization, as it enables us to correlate feature patterns between passing and failing executions across the complete codebase, rather than being limited to only the subset of code executed by failing tests.

3.4 Analysis

To address RQ1, we conducted a statistical analysis of the collected execution features. We computed the correlation of each feature with the presence of failures using TARANTULA [20], OCHIAI [2], D* [60], NAISH2 [41], and GP13 [62] coefficients across all feature types (e.g., lines, branches). These five coefficients were selected because they represent diverse statistical approaches to fault localization and have been shown to perform well in comparative studies [41, 60, 62], providing a comprehensive evaluation of feature effectiveness across different mathematical formulations of suspiciousness. For each feature type, we calculated three metrics: (1) the highest correlation among features, (2) the mean correlation of all features, and (3) the median correlation of all features. These metrics provide a comprehensive view of the suspiciousness, i.e., how likely each feature type is to correlate with the presence of failures. For instance, we have line coverage features for two subjects. For the first subject, the correlations are 0.8, 0.6, 0.4, and 0.2, respectively, for Lines 1, 2, 3, and 4. For the second subject, the correlations are 0.7, 0.5, and 0.3 for Lines 1, 2, and 3, respectively. To determine the average correlation metrics for the line feature class: the highest average correlation is $(0.8 + 0.7)/2 = 0.75$; the mean correlation across all features is $((0.8 + 0.6 + 0.4 + 0.2)/4 + (0.7 + 0.5 + 0.3)/3)/2 = 0.5$; and the median correlation is $(0.5 + 0.5)/2 = 0.5$.

Additionally, we calculate the actual correlation between the features and the presence of failures by relating the number of failing runs for which a feature is satisfied to the total number of runs for which the feature is satisfied. We utilize the *Spearman's rank correlation coefficient (Spearman's ρ)* to measure the relationship between

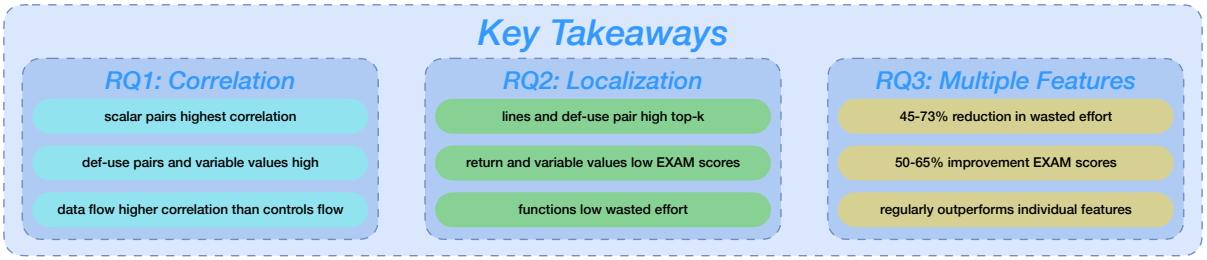


Fig. 4. Key Takeaways. Summary of the main findings regarding the suspiciousness of execution features and their relevance to fault localization.

the feature satisfaction and failure presence. This relation will show how well the feature correlates with the presence of failures, i.e., if a feature is primarily satisfied when the run is failing, which we will evaluate using Spearman's rank correlation coefficient. Similar to the similarity coefficients, we calculate the highest, mean, and median correlation when considering each subject individually for each feature type. Additionally, we calculate the overall correlation across all subjects.

We evaluated fault localization for RQ2 by applying the introduced similarity coefficients to each feature and ranking suggested lines based on faulty lines extracted from patch files in the Tests4Py dataset. While these patch files may not exclusively contain lines that directly address the fault (since fixes may involve code outside the directly faulty lines), we chose this approach to remain consistent with established methods in prior work [2, 19, 20, 26, 45, 57, 61]. Moreover, the analysis of RQ1 mitigates the potential bias introduced using the patch files for fault localization.

Following Widyasari et al. [57] and Pearson et al. [45], we assessed localization performance using the top-1, top-5, and top-10 lines suggested by each coefficient [28]. We calculated the EXAM score [59] and *wasted effort* [63] for each feature type, guided by evaluation methods used in previous studies [22, 35]. The EXAM score for a statement s is defined as:

$$\text{EXAM}_s = \frac{\text{rank}(s)}{\text{Total number of statements}} \tag{1}$$

where rank(s) is the rank of the statement s in the list of suggested lines ordered by the suspiciousness of each line. If multiple lines have the same suspiciousness, we consider the average rank of these lines, calculated as $\frac{n}{2} + (k-1)$ ([45, 53, 57]), where n is the number of lines with the same suspiciousness as s and k is the position of the first line with the same suspiciousness, i.e., the highest possible rank. The wasted effort for a line s is defined as the number of lines that need to be inspected to find the target line, starting from the top of the list of suggested lines and including the target line itself, as well as the lines with the same suspiciousness. We calculated the average over all subjects for each feature type and metric. Similar to Widyasari et al. [57] and Pearson et al. [45] we included three debugging scenarios, (1) finding one, (2) half, and (3) all faulty lines named (1) *best-case*, (2) *average-case*, and (3) *worst-case* debugging respectively.

We considered the same similarity coefficients for the localization to answer RQ3. However, we calculated the suspiciousness for each line based on the features that map to the line. We considered two cases for the multi-feature technique: one in which we calculate the maximum suspiciousness of a line for each feature, and one in which we calculate the mean suspiciousness of the features that map to the line.

3.5 Implementation

We implemented the empirical study using the fault localization tool SFLKit [50], a versatile framework supporting a range of statistical fault localization techniques, including TARANTULA [20], OCHIAI [2], D* [60], NAISH2 [41], and GP13 [62]. SFLKit provides an interface that allows us to integrate custom execution features into its pipeline, instrumenting the program to capture events during execution and offering the necessary infrastructure to analyze these events and derive features. We have constructed a workflow that instruments the program, extracts the events, assembles the features, and analyzes their effectiveness for all three research questions, providing intermediate results after each step. With its intermediate results, this process is ideal for reproducing all our findings and verifying that all steps work as intended.

Table 2. Suspiciousness and Correlation. The calculated suspiciousness for each metric TARANTULA, OCHIAI, D*, NAISH2, and GP13 and the correlation. Highlighted values represent the best results compared to the other features (comparing individual coefficients). Underlined values for the correlation represent statistical significance ($p < 0.05$).

Feature	Metric	Suspiciousness			Correlation			
		Best	Mean	Median	Overall	Best	Mean	Median
Lines	TARANTULA	0.930	**0.387**	**0.412**	0.473	1.000	0.791	0.836
	OCHIAI	0.809	**0.246**	**0.261**				
	D*	8.009	0.797	**0.708**				
	NAISH2	1.712	**0.630**	**0.742**				
	GP13	2.504	**1.372**	**1.552**				
Branches	TARANTULA	0.893	0.179	0.088	0.624	**1.000**	0.673	0.757
	OCHIAI	0.755	0.123	0.073				
	D*	7.923	0.455	0.156				
	NAISH2	1.628	0.276	0.216				
	GP13	2.399	0.603	0.383				
Functions	TARANTULA	0.863	0.300	0.305	0.504	**1.000**	0.615	0.713
	OCHIAI	0.708	0.207	0.204				
	D*	7.786	**0.805**	0.268				
	NAISH2	1.596	0.467	0.451				
	GP13	2.319	1.023	1.039				
Function Errors	TARANTULA	0.570	0.025	0.000	0.679	**1.000**	0.484	0.524
	OCHIAI	0.501	0.022	0.000				
	D*	3.153	0.175	0.000				
	NAISH2	0.959	0.051	-0.000				
	GP13	1.537	0.082	0.000				
Def-Use Pairs	TARANTULA	0.929	0.278	0.266	0.520	**1.000**	0.617	0.679
	OCHIAI	0.823	0.197	0.184				
	D*	8.100	0.729	0.280				
	NAISH2	1.695	0.451	0.417				
	GP13	2.508	0.960	0.935				
Loops	TARANTULA	0.685	0.112	0.011	0.620	**1.000**	0.569	0.696
	OCHIAI	0.527	0.076	0.008				
	D*	6.601	0.293	0.018				
	NAISH2	1.275	0.159	0.013				
	GP13	1.880	0.356	0.028				
Conditions	TARANTULA	0.888	0.175	0.078	0.607	**1.000**	0.662	0.749
	OCHIAI	0.756	0.121	0.064				
	D*	7.931	0.453	0.139				
	NAISH2	1.621	0.274	0.197				
	GP13	2.409	0.593	0.346				
Scalar Pairs	TARANTULA	**0.952**	0.184	0.043	0.845	**1.000**	0.800	0.871
	OCHIAI	**0.875**	0.120	0.039				
	D*	**8.222**	0.473	0.044				
	NAISH2	**1.745**	0.276	0.059				
	GP13	**2.604**	0.622	0.139				

3.6 Results

All summarized results of our empirical study are presented in Tables 2, 3 and 5 to 7. Tables 2 and 3 show the correlation of the execution features with the presence of failures for the metrics TARANTULA, OCHIAI, D*, NAISH2, GP13, and the corresponding Spearman's ρ values for a feature. Moreover, Tables 5 and 6 show the localization of the faults for the metrics TARANTULA, OCHIAI, D*, NAISH2, and GP13 according to top-1, top-5, top-10, EXAM score, and wasted effort for all features. Finally, Table 7 presents the results when considering a naïve multi-feature technique, as introduced in Section 2.5. The results of our empirical study are summarized in Figure 4, which highlights the key findings regarding the correlation between execution features and failures, as well as their relevance to fault localization.

3.6.1 ***RQ1: Correlation with Failures.*** To address RQ1, we conducted a comprehensive analysis of the correlation between execution features and the presence of failures, using the TARANTULA, OCHIAI, D*, NAISH2, and GP13 coefficients along with Spearman's ρ values across our extensive dataset.

Generally, a high suspiciousness score indicates a likely causality between a feature and the presence of failures. In contrast, high correlation coefficients suggest a strong relationship between the feature and the presence of failures. In other words, a feature is satisfied when a failure occurs, and the failure occurs when the feature is present. If both values are high, the feature is likely to indicate the presence of failures. In contrast, if the suspiciousness score is low, the feature is not commonly satisfied when a failure occurs. If the correlation coefficient is low, the feature is often satisfied when a failure does not occur. It is essential to note that we never compare suspiciousness scores across different coefficients (e.g., TARANTULA vs. OCHIAI), but only within the same coefficient family. Therefore, normalization across different metrics is not required and would not be meaningful for our analysis.

Table 3. Suspiciousness and Correlation. Continuation of Table 2

Feature	Metric	Suspiciousness			Correlation			
		Best	Mean	Median	Overall	Best	Mean	Median
Variable Values	TARANTULA	0.801	0.181	0.076	0.709	**1.000**	0.782	0.835
	OCHIAI	0.643	0.120	0.066				
	D*	6.910	0.435	0.119				
	NAISH2	1.466	0.288	0.215				
	GP13	2.246	0.628	0.362				
Return Values	TARANTULA	0.827	0.141	0.036	0.650	**1.000**	0.619	0.700
	OCHIAI	0.670	0.097	0.037				
	D*	6.890	0.312	0.038				
	NAISH2	1.460	0.153	0.043				
	GP13	2.173	0.423	0.114				
Null Values	TARANTULA	0.870	0.216	0.154	0.767	**1.000**	**0.841**	**0.892**
	OCHIAI	0.714	0.139	0.108				
	D*	7.610	0.368	0.193				
	NAISH2	1.614	0.360	0.312				
	GP13	2.348	0.772	0.615				
Lengths	TARANTULA	0.893	0.129	0.000	0.844	**1.000**	0.795	0.852
	OCHIAI	0.766	0.084	0.000				
	D*	7.965	0.296	0.000				
	NAISH2	1.642	0.196	0.000				
	GP13	2.419	0.440	0.000				
Empty Strings	TARANTULA	0.433	0.005	0.000	0.692	**1.000**	0.500	0.578
	OCHIAI	0.279	0.004	0.000				
	D*	4.835	0.030	0.000				
	NAISH2	0.769	0.011	0.000				
	GP13	1.295	0.020	0.000				
ASCII Strings	TARANTULA	0.816	0.107	0.077	0.898	**1.000**	0.808	0.883
	OCHIAI	0.638	0.066	0.037				
	D*	7.580	0.249	0.025				
	NAISH2	1.513	0.146	0.020				
	GP13	2.249	0.349	0.171				
Digit Strings	TARANTULA	0.689	0.032	0.000	0.862	**1.000**	0.713	0.816
	OCHIAI	0.508	0.019	0.000				
	D*	6.467	0.074	0.000				
	NAISH2	1.278	0.046	0.000				
	GP13	2.039	0.106	0.000				
Special Strings	TARANTULA	0.782	0.086	0.034	**0.915**	**1.000**	0.803	0.868
	OCHIAI	0.609	0.052	0.021				
	D*	7.085	0.236	0.021				
	NAISH2	1.461	0.116	0.011				
	GP13	2.222	0.279	0.076				
Empty Bytes	TARANTULA	0.061	0.000	0.000	0.502	**1.000**	0.065	0.000
	OCHIAI	0.039	0.000	0.000				
	D*	0.030	0.000	0.000				
	NAISH2	0.074	0.000	0.000				
	GP13	0.136	0.001	0.000				

Feature Collection Statistics. Before analyzing the correlation results, we provide context about the fine-grained nature of our execution features and their relationship to program size. Table 4 presents the average number of features collected per feature type across all subjects in our study.

The results reveal significant variations in feature density across different types. *Scalar pairs* generate the most features (3.9M on average), reflecting the combinatorial nature of variable comparisons during execution. *Lines*

Table 4. Average Feature Collection. The table shows the average number of features collected per feature type across all subjects, demonstrating the fine-grained nature of our execution feature analysis and the relative density of different feature categories.

	Lines	Branches	Functions	Function Errors	Def-Use Pairs	Loops	Conditions	Scalar Pairs	Variable Values	Return Values	Null Values	Lengths	Empty Strings	ASCII Strings	Digit Strings	Special Strings	Empty Bytes
Mean Features	61.17K	6.55K	2.99K	0.08K	20.79K	0.87K	7.64K	3868.16K	12.38K	3.17K	16.85K	22.20K	0.21K	12.16K	3.74K	10.19K	0.01K

come in as the second most frequent feature type (62K on average), indicating that while line execution is a common event, it is less frequent than variable comparisons. *Variable values*, *null values*, and *lengths* also produce substantial numbers of features (12K-22K on average), indicating frequent variable manipulations in typical program executions. In contrast, specialized features like *empty strings* and *empty bytes* occur less frequently (0.2K and 0.01K, respectively), reflecting their specificity to specific execution patterns.

Figure 5 illustrates how feature collection scales with program size, measured in lines of code. The figure demonstrates that feature collection grows proportionally with program complexity, with larger programs naturally generating more execution events and corresponding features. This scaling relationship validates that our approach can handle programs of varying sizes while maintaining the fine-grained analysis necessary for effective fault diagnosis.

These statistics demonstrate that our approach captures execution behavior at multiple granularities: from high-frequency basic operations (lines, variables) to specialized patterns (string properties, exception handling). The fine-grained nature of feature collection, particularly for scalar pairs and variable-related features, provides the detailed execution context necessary for accurate fault diagnosis while maintaining computational feasibility across programs of different scales.

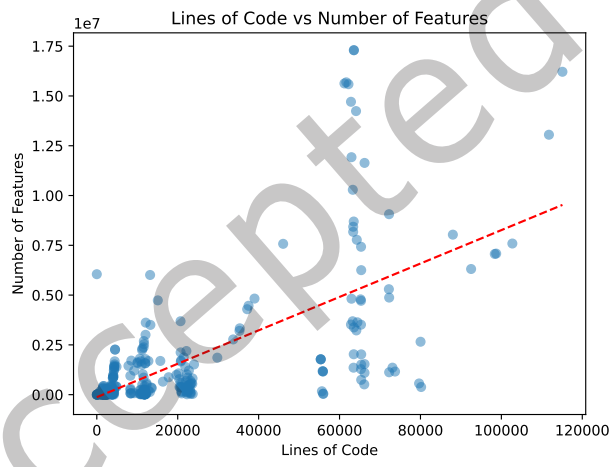


Fig. 5. Feature Collection vs. Program Size. Distribution of collected features across programs of different sizes (measured in lines of code), showing how the number of execution features scales with program complexity. The relationship demonstrates the scalability of our approach across programs of varying sizes.

Data Flow Features Dominate Failure Correlation. Our quantitative analysis reveals a striking pattern: features capturing data flow and data relationships consistently demonstrate the highest correlation with failures across all suspiciousness metrics. Among all features, *special strings* achieve the highest overall correlation with 0.92, indicating that strings containing non-alphanumeric characters are particularly indicative of faults. *Scalar pairs* follow closely with Spearman's ρ values ranging from 0.80 to 0.87 across different metrics, indicating a robust statistical relationship with fault occurrence. *Null values* and *variable values* also show high correlation values of 0.84–0.89 and 0.78–0.84 respectively. *Def-use pairs* also show high suspiciousness scores, with correlation values of 0.62–0.68, suggesting that the relationships between variable definitions and their uses are also highly relevant to fault manifestation.

It is important to note that while string properties (particularly special strings) achieve the highest correlation coefficients, their practical applicability is limited by their coverage, i.e., they only occur in executions that process string data. Specialized features, such as string properties, are less frequent across program executions compared to more universal features, like scalar pairs or def-use pairs, which can be observed in almost any execution involving variable comparisons or data flow. This distribution explains why, despite their high correlation when present, they may not appear as prominently in broader fault localization analyses, where coverage and frequency are important considerations.

This quantitative dominance of data flow features suggests that fault manifestation is fundamentally tied to data propagation and transformation patterns rather than structural execution paths.

Control Flow vs. Data Flow: A Quantitative Comparison. While features representing control flow, including branches, conditions, and loops, show moderate correlations (Spearman's ρ values of 0.57-0.76), they consistently underperform compared to data flow features. This performance gap becomes more pronounced when examining the distributions of the suspiciousness coefficients.

*Features that capture data and control flow, particularly **string properties** and **scalar pairs**, are highly correlated with failures. However, the practical utility of features depends on both their correlation strength and their coverage across executions.*

Additionally, *Lines* show a high correlation with the presence of failures, particularly when considering the best feature of each class, but also on the mean and median of all features, compared with the other feature classes. While *functions* have slightly lower correlations than lines, they still show significant correlations for the mean and median of all features. However, these features cover almost the entire program, with exceptionally executed functions corresponding to an entire code block. Due to this information overload, they may not be specific enough to pinpoint the causes of a fault. Note that the cause of the fault may differ from the location of the fault. This finding is supported by Spearman's ρ values, which show that lines and functions are less correlated with failures.

***Lines** and **functions** show a high suspicion for the occurrence of failures but are not specific enough to accurately detect a fault's causes based on Spearman's ρ values.*

While *return values* provide a high suspiciousness, like other value-related features, they do not show a high correlation with the presence of failures, suggesting that they might not be as relevant to failure occurrences as the variables themselves and their properties, e.g., properties corresponding to well-known failure patterns, for instance, null pointers or index-out-of-bounds.

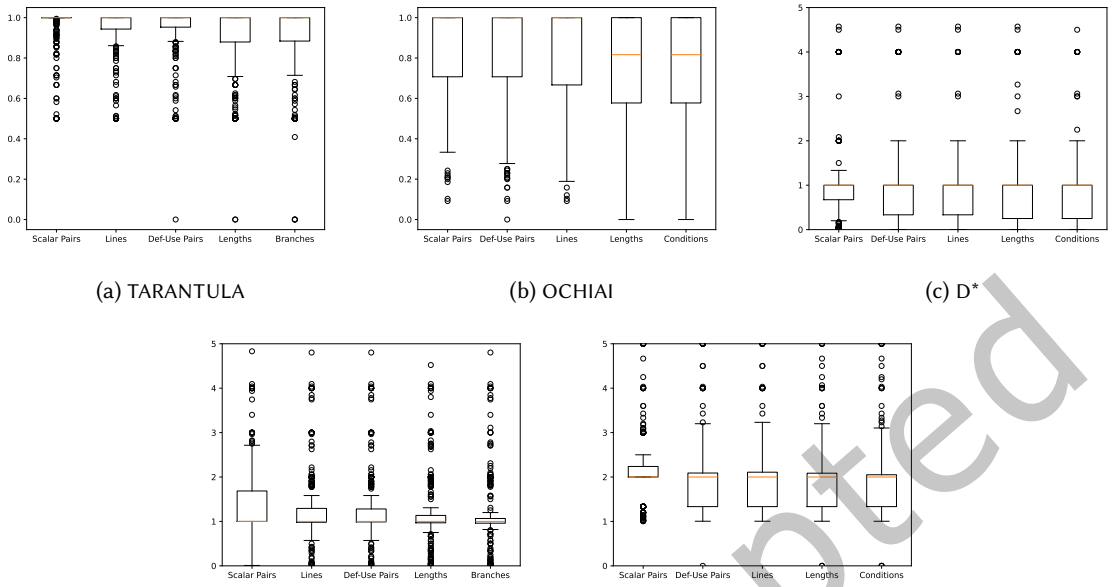
Counterintuitively, features that capture if a function terminates with exceptions do *not* highly correlate with failure occurrence, suggesting they might not be as relevant.

Features that track if a function exits with exceptions do not highly correlate with failure occurrence.

Figure 6 illustrates the suspiciousness results for the five best-performing execution features leveraging TARANTULA, OCHIAI, D*, NAISH2, and GP13 for each subject. These results support our findings that the features that capture data flow and the relation between a program's data, specifically *scalar pairs*, *def-use pairs*, and *length of objects*, consistently achieve the highest suspiciousness scores across all metrics. Note that while string properties exhibit the highest correlation coefficients in our analysis, they appear less frequently in this best-performing features analysis due to their limited coverage, as they are only applicable to executions involving string processing. In addition, we can see the dominance of *lines* and *branches/conditions* in the suspiciousness scores. However, as our deeper analysis reveals, branches and conditions do not exhibit a high correlation with the presence of failures, suggesting that they can indicate the effect of a failure but may not reveal the root cause. In contrast, lines are more strongly correlated with failures, indicating that they are more likely to be involved in the actual fault, which is unsurprising since they cover the entire codebase.

Lines remain a valuable feature for fault localization.

3.6.2 ***RQ2: Localizing Faults.*** Concerning the fault localization and addressing RQ2, we can see in Tables 5 and 6 that *lines* outperform all other feature classes in localizing faulty lines for the top-1 metric.



(a) TARANTULA (b) OCHIAI (c) D*

(d) NAISH2 (e) GP13

Fig. 6. Suspiciousness. Results of the suspiciousness that a feature correlates with the presence of failures for the metrics TARANTULA, OCHIAI, D*, NAISH2, and GP13. The figures display the top five features for each metric. Each feature is evaluated according to the best feature of each class for each subject.

Lines Excel in Early Fault Detection But Show Limitations at Scale. The quantitative analysis reveals *lines* achieve exceptional top-1 performance, with success rates of around 14% across most suspiciousness metrics, substantially outperforming other individual features. However, this early detection advantage diminishes as the scope of the inspection expands. While lines maintain strong top-5 performance (ca. 24% success rate), the performance gap narrows significantly at top-10 (around 27%), where *def-use pairs* achieve a comparable success rate for top-5 (around 25%) and outperform lines at top-10 (around 30%). This pattern suggests that while lines provide immediate fault indication, their discriminative power decreases when developers must examine larger code sections.

***Lines** excel in early fault detection while **def-use pairs** are more effective for broader inspection scopes.*

Functions vs. Lines: A Trade-off Between Precision and Coverage. For the EXAM scores, we see that *functions* show the lowest score. However, lines, def-use pairs, and function errors also show a comparable low EXAM score, indicating that these features are more likely to suggest the faulty lines early in the suggested lines. The wasted effort reinforces the results for the EXAM score, where lines, def-use pairs, functions, and function errors show the lowest score. The superior EXAM performance of functions (0.087–0.101 across metrics) reflects their ability to capture fault-containing code regions with greater granularity, thereby reducing the likelihood of developers missing fault locations due to an overly narrow focus. This trade-off becomes critical in real-world debugging, where fault manifestation may involve interactions across multiple statements within a function boundary.

Table 5. Localization. Results for the localization of faults leveraging TARANTULA, OCHIAI, D*, NAISH2, and GP13. Each feature is evaluated according to Top-1, Top-5, Top-10, EXAM score, and wasted effort for three debugging scenarios.

Feature	Metric	Best-Case Debugging					Average-Case Debugging					Worst-Case Debugging				
		Top-k			EXAM	Effort	Top-k			EXAM	Effort	Top-k			EXAM	Effort
		1	5	10			1	5	10			1	5	10		
Lines	TARANTULA	**14.2%**	25.4%	29.5%	0.101	6.1k	**9.8%**	18.5%	21.7%	0.169	10.8k	**7.6%**	13.9%	16.0%	0.316	29.3k
	OCHIAI	**14.0%**	25.0%	28.7%	0.101	6.1k	**10.0%**	18.0%	21.0%	0.169	10.8k	**7.7%**	13.8%	15.6%	0.316	29.3k
	D*	**9.2%**	14.9%	18.6%	0.118	6.6k	**6.1%**	9.5%	13.3%	0.185	11.2k	**4.5%**	6.9%	9.6%	0.329	29.7k
	NAISH2	**14.4%**	23.7%	26.5%	0.102	6.1k	**10.1%**	17.2%	19.8%	0.170	10.8k	**7.8%**	13.2%	15.0%	0.316	29.3k
	GP13	**14.5%**	23.6%	26.5%	0.102	6.1k	**10.1%**	17.1%	19.9%	0.170	10.8k	**7.8%**	13.1%	15.1%	0.317	29.3k
Branches	TARANTULA	8.0%	17.6%	21.4%	0.244	16.3k	4.8%	12.6%	16.1%	0.291	22.4k	3.4%	9.3%	12.0%	0.392	37.4k
	OCHIAI	7.7%	17.1%	21.0%	0.244	16.3k	4.7%	12.3%	15.7%	0.292	22.5k	3.3%	9.1%	11.7%	0.392	37.5k
	D*	4.6%	11.8%	15.3%	0.250	16.4k	2.6%	7.8%	10.6%	0.297	22.5k	1.8%	5.9%	7.6%	0.396	37.5k
	NAISH2	7.2%	15.9%	19.3%	0.245	16.3k	4.6%	11.7%	14.6%	0.292	22.4k	3.3%	8.9%	11.2%	0.392	37.4k
	GP13	7.3%	15.9%	19.1%	0.245	16.4k	4.7%	11.7%	14.5%	0.292	22.5k	3.3%	8.8%	11.1%	0.392	37.5k
Functions	TARANTULA	6.2%	20.8%	27.0%	**0.087**	**5.4k**	3.9%	15.3%	20.5%	**0.107**	**6.0k**	2.7%	11.1%	14.8%	**0.223**	**20.7k**
	OCHIAI	6.1%	20.2%	26.6%	**0.087**	**5.4k**	3.9%	15.1%	20.4%	**0.107**	**6.0k**	2.8%	11.2%	15.0%	**0.222**	**20.7k**
	D*	5.2%	14.0%	20.4%	**0.101**	**5.7k**	3.4%	10.0%	14.9%	**0.120**	**6.3k**	2.4%	7.4%	10.8%	**0.230**	**20.9k**
	NAISH2	6.1%	19.5%	25.2%	**0.087**	**5.3k**	4.0%	14.9%	19.7%	**0.106**	**5.9k**	2.8%	11.0%	14.7%	**0.222**	**20.7k**
	GP13	6.0%	19.6%	25.5%	**0.088**	**5.4k**	4.0%	15.0%	19.8%	**0.107**	**6.0k**	2.8%	11.1%	14.8%	**0.222**	**20.7k**
Function Errors	TARANTULA	3.4%	11.2%	16.2%	0.123	7.0k	2.3%	7.7%	11.6%	0.142	7.6k	1.5%	5.8%	8.4%	0.244	21.9k
	OCHIAI	3.5%	11.2%	16.2%	0.123	7.0k	2.3%	7.8%	11.7%	0.142	7.6k	1.6%	5.8%	8.5%	0.244	21.9k
	D*	1.5%	6.0%	9.2%	0.146	7.6k	0.7%	3.5%	5.9%	0.163	8.1k	0.5%	2.7%	4.4%	0.261	22.4k
	NAISH2	3.4%	10.8%	15.6%	0.121	6.9k	2.3%	7.8%	11.5%	0.142	7.5k	1.6%	5.7%	8.3%	0.246	21.9k
	GP13	3.4%	10.9%	15.9%	0.123	7.0k	2.4%	7.7%	11.4%	0.142	7.6k	1.6%	5.7%	8.3%	0.244	21.9k
Def-Use Pairs	TARANTULA	8.5%	**25.9%**	**32.9%**	0.120	10.6k	5.6%	**19.2%**	**24.9%**	0.187	13.2k	4.1%	**14.8%**	**18.7%**	0.344	37.5k
	OCHIAI	8.6%	**25.7%**	**31.9%**	0.120	10.6k	5.8%	**19.2%**	**24.2%**	0.187	13.2k	4.3%	**14.9%**	**18.5%**	0.344	37.5k
	D*	6.1%	**14.9%**	**22.1%**	0.128	10.8k	4.0%	**10.0%**	**16.2%**	0.194	13.3k	2.9%	**7.7%**	**12.4%**	0.349	37.6k
	NAISH2	8.4%	**24.1%**	**29.1%**	0.120	10.6k	5.8%	**18.7%**	**22.9%**	0.187	13.1k	4.2%	**14.5%**	**17.9%**	0.343	37.5k
	GP13	8.3%	**24.3%**	**29.5%**	0.121	10.7k	5.8%	**18.7%**	**22.9%**	0.188	13.2k	4.2%	**14.5%**	**17.8%**	0.344	37.5k
Loops	TARANTULA	3.6%	8.6%	8.7%	0.407	27.3k	1.3%	5.7%	6.1%	0.432	32.5k	0.8%	3.7%	4.2%	0.467	45.0k
	OCHIAI	3.6%	8.5%	9.0%	0.407	27.3k	1.3%	5.7%	6.4%	0.432	32.5k	0.8%	3.7%	4.5%	0.466	45.0k
	D*	2.4%	6.6%	7.2%	0.407	27.3k	0.9%	4.1%	4.8%	0.432	32.5k	0.6%	2.7%	3.3%	0.467	45.0k
	NAISH2	3.6%	8.4%	8.6%	0.407	27.3k	1.3%	5.4%	5.8%	0.432	32.5k	0.8%	3.6%	4.2%	0.466	45.0k
	GP13	3.5%	8.4%	8.6%	0.407	27.3k	1.2%	5.4%	5.8%	0.432	32.5k	0.8%	3.6%	4.2%	0.466	45.0k
Conditions	TARANTULA	6.9%	13.6%	16.2%	0.349	29.3k	4.0%	8.0%	9.3%	0.433	41.2k	2.8%	5.6%	6.3%	0.476	45.5k
	OCHIAI	6.3%	13.1%	15.9%	0.349	29.3k	3.8%	7.8%	9.3%	0.433	41.2k	2.7%	5.6%	6.3%	0.476	45.5k
	D*	3.8%	9.5%	12.8%	0.350	29.3k	1.8%	5.3%	7.2%	0.433	41.2k	1.1%	3.5%	4.8%	0.476	45.5k
	NAISH2	5.9%	11.8%	14.5%	0.349	29.3k	3.7%	7.3%	8.9%	0.433	41.2k	2.7%	5.3%	6.1%	0.476	45.5k
	GP13	6.0%	11.7%	14.4%	0.349	29.3k	3.8%	7.2%	8.9%	0.433	41.2k	2.7%	5.3%	6.2%	0.476	45.5k
Scalar Pairs	TARANTULA	5.7%	14.3%	15.6%	0.254	20.2k	3.3%	8.3%	9.4%	0.343	26.5k	2.2%	5.7%	6.1%	0.439	42.4k
	OCHIAI	5.8%	13.7%	15.2%	0.254	20.2k	3.7%	8.1%	9.5%	0.343	26.5k	2.5%	5.5%	6.3%	0.439	42.4k
	D*	4.0%	8.8%	9.9%	0.258	20.4k	2.2%	4.8%	5.7%	0.345	26.6k	1.6%	3.4%	3.9%	0.440	42.4k
	NAISH2	5.9%	12.7%	14.3%	0.255	20.2k	3.7%	7.8%	9.3%	0.343	26.5k	2.5%	5.5%	6.4%	0.439	42.4k
	GP13	5.9%	12.7%	14.3%	0.255	20.2k	3.7%	7.9%	9.3%	0.343	26.5k	2.5%	5.5%	6.5%	0.439	42.4k

Feature Coverage Bias and Its Implications. However, some features are inherently better suited by design for localizing faults. For instance, lines and functions covering almost the entire program are more likely to assign reasonable suspiciousness scores to the faulty lines and rank them higher. Other features can only partially cover the code and may not assign a reasonable suspiciousness score to the actual faulty lines. For instance, loops only cover a small portion of the code and might not be executed in the presence of a failure. This coverage bias is quantitatively evident: while *lines* and *functions* maintain presence across all program executions, specialized features like *string properties* are less frequent.

By design, some features are better suited for localizing faults than others.

Table 6. Localization. Continuation of Table 5.

Feature	Metric	Best-Case Debugging					Average-Case Debugging					Worst-Case Debugging				
		Top-k			EXAM	Effort	Top-k			EXAM	Effort	Top-k			EXAM	Effort
		1	5	10			1	5	10			1	5	10		
Variable Values	TARANTULA	2.7%	5.6%	7.1%	0.433	41.7k	1.3%	2.8%	4.3%	0.467	45.4k	1.0%	1.9%	3.2%	0.482	48.7k
	OCHIAI	2.7%	5.3%	7.1%	0.433	41.7k	1.5%	2.7%	4.1%	0.467	45.4k	1.1%	1.9%	3.1%	0.482	48.7k
	D*	1.7%	4.3%	6.0%	0.433	41.7k	0.9%	2.2%	3.5%	0.467	45.4k	0.7%	1.6%	2.7%	0.482	48.7k
	NAISH2	2.5%	4.5%	6.5%	0.433	41.7k	1.3%	2.1%	3.7%	0.467	45.4k	1.0%	1.5%	2.8%	0.482	48.7k
	GP13	2.5%	4.5%	6.5%	0.433	41.7k	1.3%	2.1%	3.7%	0.467	45.4k	1.0%	1.5%	2.8%	0.482	48.7k
Return Values	TARANTULA	6.1%	8.6%	10.2%	0.433	45.4k	4.5%	6.1%	6.6%	0.461	48.3k	3.6%	4.6%	4.8%	0.487	49.5k
	OCHIAI	6.2%	8.5%	9.9%	0.433	45.4k	4.7%	5.9%	6.5%	0.461	48.3k	3.7%	4.5%	4.7%	0.487	49.5k
	D*	3.5%	6.8%	9.3%	0.434	45.4k	2.5%	4.7%	6.1%	0.461	48.3k	2.0%	3.5%	4.5%	0.487	49.5k
	NAISH2	6.2%	8.5%	9.7%	0.433	45.4k	4.7%	5.8%	6.5%	0.461	48.3k	3.7%	4.5%	4.7%	0.487	49.5k
	GP13	6.2%	8.4%	9.6%	0.433	45.4k	4.7%	5.9%	6.4%	0.461	48.3k	3.7%	4.5%	4.7%	0.487	49.5k
Null Values	TARANTULA	6.0%	10.0%	11.6%	0.256	20.3k	3.4%	5.5%	6.7%	0.342	26.5k	2.3%	3.9%	4.6%	0.438	42.4k
	OCHIAI	6.1%	10.0%	11.4%	0.256	20.3k	3.5%	5.7%	6.6%	0.342	26.5k	2.3%	4.0%	4.6%	0.438	42.4k
	D*	3.3%	6.3%	8.3%	0.258	20.4k	1.3%	3.4%	4.5%	0.344	26.6k	0.8%	2.5%	3.2%	0.439	42.4k
	NAISH2	6.1%	9.4%	10.4%	0.255	20.3k	3.5%	5.5%	6.3%	0.341	26.5k	2.4%	4.0%	4.5%	0.438	42.4k
	GP13	6.1%	9.5%	10.4%	0.256	20.3k	3.5%	5.5%	6.3%	0.342	26.5k	2.4%	3.9%	4.4%	0.438	42.4k
Lengths	TARANTULA	7.2%	12.5%	15.0%	0.332	28.1k	3.8%	7.9%	8.8%	0.415	38.6k	2.4%	5.4%	5.7%	0.470	47.9k
	OCHIAI	7.4%	12.1%	14.7%	0.332	28.1k	4.2%	7.7%	8.7%	0.415	38.6k	2.6%	5.4%	5.7%	0.470	47.9k
	D*	4.1%	6.7%	8.9%	0.334	28.2k	1.7%	3.7%	4.9%	0.416	38.6k	1.1%	2.7%	3.1%	0.471	47.9k
	NAISH2	7.7%	12.0%	13.9%	0.332	28.2k	4.2%	8.0%	8.5%	0.415	38.6k	2.6%	5.6%	5.7%	0.470	47.9k
	GP13	7.8%	11.9%	13.9%	0.332	28.2k	4.2%	7.9%	8.6%	0.415	38.6k	2.6%	5.6%	5.7%	0.470	47.9k
Empty Strings	TARANTULA	1.6%	3.3%	5.5%	0.262	20.7k	0.7%	1.7%	3.2%	0.344	26.8k	0.4%	1.3%	2.3%	0.440	42.5k
	OCHIAI	1.6%	3.2%	5.3%	0.262	20.7k	0.7%	1.7%	3.1%	0.344	26.8k	0.4%	1.3%	2.3%	0.440	42.5k
	D*	1.0%	3.2%	5.3%	0.262	20.7k	0.3%	1.7%	3.1%	0.345	26.8k	0.2%	1.3%	2.3%	0.440	42.5k
	NAISH2	1.6%	3.2%	5.3%	0.262	20.7k	0.7%	1.7%	3.1%	0.344	26.8k	0.4%	1.3%	2.3%	0.440	42.5k
	GP13	1.6%	3.2%	5.3%	0.262	20.7k	0.7%	1.7%	3.1%	0.344	26.8k	0.4%	1.3%	2.3%	0.440	42.5k
ASCII Strings	TARANTULA	6.7%	10.2%	11.0%	0.259	20.6k	3.9%	6.5%	6.9%	0.344	26.7k	2.5%	4.7%	4.9%	0.439	42.5k
	OCHIAI	6.5%	9.8%	10.8%	0.259	20.6k	4.2%	6.3%	6.8%	0.344	26.7k	2.7%	4.6%	4.8%	0.439	42.5k
	D*	3.6%	6.1%	8.0%	0.260	20.6k	2.2%	3.4%	4.7%	0.345	26.7k	1.6%	2.7%	3.3%	0.440	42.5k
	NAISH2	6.2%	9.6%	10.9%	0.259	20.6k	4.1%	6.2%	7.0%	0.344	26.7k	2.6%	4.7%	5.0%	0.439	42.5k
	GP13	6.1%	9.5%	10.7%	0.259	20.6k	4.0%	6.2%	6.9%	0.344	26.7k	2.6%	4.7%	5.0%	0.439	42.5k
Digit Strings	TARANTULA	3.3%	6.7%	7.9%	0.260	20.7k	1.7%	4.3%	5.2%	0.344	26.7k	1.1%	3.2%	3.7%	0.439	42.5k
	OCHIAI	3.3%	6.6%	7.9%	0.260	20.7k	1.8%	4.2%	5.1%	0.344	26.7k	1.2%	3.2%	3.6%	0.439	42.5k
	D*	1.6%	4.4%	6.3%	0.261	20.7k	0.5%	2.5%	3.8%	0.344	26.8k	0.4%	1.8%	2.8%	0.439	42.5k
	NAISH2	2.9%	6.5%	7.7%	0.260	20.7k	1.7%	4.1%	4.9%	0.344	26.7k	1.1%	3.1%	3.5%	0.439	42.5k
	GP13	2.9%	6.5%	7.7%	0.260	20.7k	1.7%	4.2%	4.9%	0.344	26.7k	1.1%	3.0%	3.4%	0.439	42.5k
Special Strings	TARANTULA	6.7%	9.5%	9.6%	0.260	20.6k	3.9%	6.0%	6.0%	0.344	26.7k	2.5%	4.3%	4.6%	0.439	42.5k
	OCHIAI	6.6%	9.3%	9.6%	0.260	20.6k	4.0%	5.7%	6.1%	0.344	26.7k	2.7%	4.1%	4.6%	0.439	42.5k
	D*	4.2%	6.6%	7.6%	0.260	20.7k	2.4%	3.7%	4.5%	0.345	26.8k	1.7%	2.8%	3.3%	0.440	42.5k
	NAISH2	6.5%	9.1%	9.8%	0.259	20.6k	4.0%	5.7%	6.1%	0.344	26.7k	2.7%	4.2%	4.5%	0.439	42.5k
	GP13	6.5%	9.1%	9.6%	0.260	20.6k	4.1%	5.7%	6.0%	0.344	26.7k	2.7%	4.1%	4.5%	0.439	42.5k
Empty Bytes	TARANTULA	0.6%	3.0%	5.4%	0.262	20.7k	0.3%	1.8%	3.2%	0.344	26.8k	0.3%	1.3%	2.3%	0.439	42.5k
	OCHIAI	0.6%	3.0%	5.4%	0.262	20.7k	0.4%	1.8%	3.2%	0.344	26.8k	0.3%	1.3%	2.3%	0.439	42.5k
	D*	0.6%	3.1%	5.4%	0.262	20.7k	0.3%	1.8%	3.2%	0.344	26.8k	0.3%	1.3%	2.3%	0.439	42.5k
	NAISH2	0.6%	3.0%	5.4%	0.262	20.7k	0.4%	1.8%	3.2%	0.344	26.8k	0.3%	1.3%	2.3%	0.439	42.5k
	GP13	0.6%	3.0%	5.4%	0.262	20.7k	0.3%	1.8%	3.1%	0.344	26.8k	0.3%	1.3%	2.3%	0.439	42.5k

3.6.3 ***RQ3: Multi-Feature Localization.*** Our evaluation of the multi-feature technique for RQ3 demonstrates significant advantages over traditional line-only fault localization approaches, particularly in practical debugging scenarios where developer effort is paramount. Table 7 provides comprehensive quantitative evidence that multi-feature approaches consistently achieve superior performance in the most critical metrics for real-world debugging effectiveness.

Table 7. Multi-Feature Fault Localization Results. Comparison of multi-feature techniques ($Multi_{max}$ and $Multi_{mean}$) using TARANTULA, OCHIAI, D*, NAISH2, and GP13 suspiciousness metrics. Multi-feature approaches demonstrate superior performance in EXAM scores and wasted effort—the most critical metrics for practical debugging effectiveness—while maintaining competitive top-k performance. Bold values highlight where multi-feature approaches achieve the best results in each debugging scenario.

Feature	Metric	Best-Case Debugging					Average-Case Debugging					Worst-Case Debugging				
		Top-k			EXAM	Effort	Top-k			EXAM	Effort	Top-k			EXAM	Effort
		1	5	10			1	5	10			1	5	10		
$Multi_{max}$	TARANTULA	8.2%	21.2%	30.2%	**0.056**	**2.2k**	5.4%	15.1%	22.7%	**0.084**	**5.2k**	3.8%	11.4%	16.6%	**0.178**	**10.1k**
	OCHIAI	8.6%	21.1%	30.3%	**0.056**	**2.2k**	5.8%	15.3%	23.1%	**0.085**	**5.2k**	4.1%	11.7%	17.0%	**0.178**	**10.1k**
	D*	5.7%	16.3%	20.3%	**0.082**	**3.0k**	3.5%	11.6%	14.8%	**0.110**	**6.0k**	2.4%	**8.7%**	10.9%	**0.197**	**10.6k**
	NAISH2	8.5%	20.2%	28.4%	**0.060**	**2.3k**	5.7%	15.4%	22.2%	**0.087**	**5.3k**	4.1%	11.9%	16.6%	**0.179**	**10.1k**
	GP13	8.5%	20.2%	28.5%	**0.060**	**2.3k**	5.7%	15.3%	22.2%	**0.087**	**5.3k**	4.1%	11.9%	16.6%	**0.179**	**10.1k**
$Multi_{mean}$	TARANTULA	8.2%	21.1%	30.2%	0.161	6.3k	5.3%	15.1%	22.7%	0.181	9.1k	3.8%	11.4%	16.6%	0.249	13.3k
	OCHIAI	8.6%	21.1%	30.3%	0.173	6.6k	5.7%	15.3%	23.1%	0.188	9.3k	4.1%	11.7%	17.1%	0.252	13.4k
	D*	5.7%	**16.3%**	20.3%	0.173	6.6k	3.5%	**11.7%**	14.7%	0.188	9.3k	2.4%	8.7%	10.9%	0.252	13.4k
	NAISH2	8.4%	20.2%	28.3%	0.173	6.6k	5.7%	15.3%	22.2%	0.188	9.3k	4.1%	11.9%	16.6%	0.252	13.4k
	GP13	8.5%	20.2%	28.4%	0.173	6.6k	5.7%	15.3%	22.2%	0.188	9.3k	4.1%	11.9%	16.6%	0.252	13.4k

Quantified Effort Reduction Across All Metrics. The multi-feature technique substantially reduces wasted effort compared to individual feature classes across all debugging scenarios, with statistically significant improvements ranging from 45% to 73%. Specifically, $Multi_{max}$ with TARANTULA achieves wasted efforts of only 2.2k, 5.2k, and 10.1k lines for best-, average-, and worst-case debugging respectively. Comparing these results to the best-performing individual features, i.e., *lines* (6.1k, 10.8k, and 29.3k) and *functions* (5.4k, 6.0k, and 20.7k), we observe that the multi-feature approach reduces wasted effort by 65%, 52%, and 65%, respectively. These results are supported by the improvement across all suspiciousness metrics, where the multi-feature approach significantly reduces wasted effort compared to the best individual features.

Early Fault Detection: Quantitative EXAM Score Analysis. The multi-feature approaches consistently demonstrate superior EXAM scores, indicating a substantially higher likelihood of suggesting faulty lines early in the inspection process. $Multi_{max}$ achieves EXAM scores of 0.056 (best-case), 0.084 (average-case), and 0.178 (worst-case) across the three debugging scenarios. These scores represent significant improvements compared to each feature.

Top-K Performance: Balanced Excellence Across Ranking Metrics. While individual features may excel in specific top-k scenarios, the multi-feature technique consistently performs across all ranking thresholds. For Top-1 accuracy, $Multi_{max}$ achieves consistently high success rates outperforming most individual features, while only being dominated by a small set of different features across the various metrics.

Aggregation Strategy Analysis: Maximum vs. Mean Suspiciousness. Our comparative analysis reveals that the maximum suspiciousness aggregation ($Multi_{max}$) consistently outperforms the mean aggregation ($Multi_{mean}$) across all metrics and scenarios, except D* for the top-5 metric. This finding aligns with the intuition that taking the maximum suspiciousness preserves strong signals from the most discriminative features, while averaging may dilute important fault indicators.

Multi-feature approaches significantly outperform individual features in practical metrics for debugging effectiveness, reducing wasted effort and improving EXAM scores across all suspiciousness metrics.

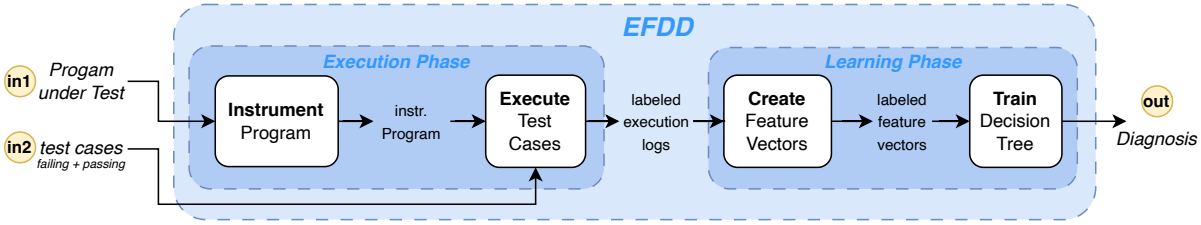


Fig. 7. EFDD at a glance. EFDD takes a program and a set of labeled test cases as input, instruments the program, executes the test cases, and captures an execution trace. From this trace, it constructs execution features to train a decision tree. The resulting model offers an interpretable diagnosis for the observed fault.

4 Learning Diagnoses from Execution Features

In software debugging, developers often face the challenge of diagnosing faults based on available test cases. EFDD (Execution-Feature Driven Debugging) addresses this by systematically transforming raw execution data into actionable insights, helping developers localize faults with greater precision and interpretability.

We focus on scenarios where developers are equipped with a faulty program and a set of labeled test cases—comprising both passing and failing cases—that effectively expose the fault. The primary objective of EFDD is to utilize these labeled test executions to infer an interpretable diagnosis that explains why specific test cases trigger failures while others succeed. At its core, EFDD leverages the contrast between passing and failing execution features to extract meaningful diagnoses. Analyzing execution traces identifies correlations between execution features and observed outcomes, enabling an execution-driven approach to fault diagnosis.

Figure 7 outlines the architecture of EFDD, which is composed of two primary phases: the Execution Phase and the Learning Phase. The process unfolds through the following sequential steps:

(1) (*Program Instrumentation*) EFDD begins by instrumenting the program under test. This instrumentation embeds probes within the code to capture execution events, allowing us to understand the program's dynamic behavior. (2) (*Test Execution*) The instrumented program is then executed using the provided test cases. Both passing and failing tests are run to gather diverse event execution traces. Each trace consists of a chronological sequence of events triggered during the program's execution, offering a fine-grained view of the program's behavior under different inputs. (3) (*Feature Extraction*) From the collected execution traces, EFDD constructs feature vectors that abstract relevant characteristics of each run. These features capture key execution properties—such as the frequency of certain events, the activation of specific code paths, or data dependencies—that may correlate with faults. The feature engineering process ensures that the resulting vectors are informative and amenable to machine learning. (4) (*Model Training*) With the labeled feature vectors, EFDD trains a decision tree classifier. This model learns to discriminate between passing and failing executions by identifying patterns in the extracted features. The choice of a decision tree ensures that the resulting diagnosis is transparent and easily understandable by developers, as the tree structure naturally maps to logical conditions that can be traced back to the program's behavior. (5) (*Diagnosis Generation*) Finally, EFDD analyzes the trained decision tree to produce a diagnosis that pinpoints the program behaviors most strongly associated with failures. The diagnosis highlights critical decision points and feature thresholds that differentiate between failing and passing runs, providing developers with clear insights into the root cause of the fault.

For any subsequent input or test, EFDD would streamline this process. It executes the input, captures the events, formulates the features, and then classifies the run based on the initially trained model.

4.1 Training a Decision Tree Model

The concluding step of our approach involves training a machine learning classifier using the feature vectors we have constructed. Inspired by ALHAZEN [23], we opt for decision trees as our classifier of choice for several reasons. First, our features predominantly consist of discrete values, making them well-suited for decision tree-based classification. These features enable us to distinguish effectively between failing and passing runs through straightforward combinations of feature values. Second, the explainable nature of decision trees enables us to identify relevant features that contribute to fault detection quickly. Finally, this model enables classifying the behavior of an unseen test input by correlating extracted execution features with program failures.

Our preliminary experiments with various classifiers, including other naive classifiers, neural networks, and large language models, consistently revealed that decision trees either outperformed or matched the performance of the best classifiers in all cases, making them ideal candidates for our classification models. Additionally, imbalanced sample sizes were not an issue, as most experiments, like those in the real world, had fewer failing than passing examples.

We are committed to maintaining impartiality in our classification process and do not favor detecting faults over identifying correct executions. We consider passing and failing executions equally valuable for learning an adequate diagnosis. Biasing our classification towards a particular outcome could lead to issues in any downstream applications of our approach.

4.2 Deriving Diagnoses

A central design goal of EFDD is to ensure the **interpretability** of its machine learning model, enabling developers to gain clear insights into the underlying causes of software faults. Using decision trees as the core diagnostic model was a deliberate choice, as they inherently provide a transparent and logical decision-making flow that developers can easily follow. By framing the diagnosis as a decision tree, EFDD transforms complex execution traces into intuitive, rule-based explanations. Each node in the tree represents a conditional check derived from the program's execution features, guiding developers through the logical paths that lead to passing or failing outcomes. This approach pinpoints fault indicators and offers a deeper understanding of the execution contexts that trigger failures. We emphasize that EFDD is not a fault localization technique, i.e., it does not rank program elements based on their likelihood of being faulty, but instead provides a high-level explanation of the conditions under which failures occur.

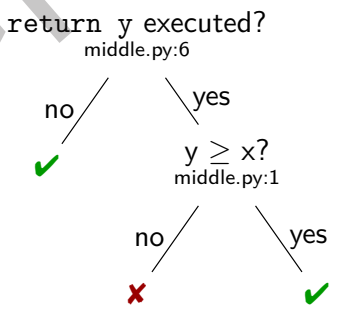


Fig. 8. The decision tree generated by EFDD for the `middle()` example. Each node represents a decision, leading to a classification of either ✔ (Pass) or ✘ (Fail).

We illustrate this interpretability through the `middle()` example introduced in Section 1. Applying EFDD to this case, using the initial set of labeled test cases, yields the decision tree shown in Figure 8.

The decision tree reveals two critical features associated with the fault: (1) **Execution of Line 6:** Whether the statement `return y` at Line 6 was executed. (2) **Comparison of Variables (y ≥ x):** Whether the value of `y` is greater than or equal to `x` when entering the function. The fault occurs under a specific condition: when Line 6 is executed and `y` is not greater than or equal to `x`. This precise insight directs developers to the faulty behavior without requiring them to comb through the entire execution trace. From the decision tree, we can infer the following: (1) If Line 6 is **not** executed, the test case passes—irrespective of other conditions. (2) If Line 6 **is** executed and $y \geq x$, the execution still passes. (3) However, if Line 6 is executed **and** $y < x$, the failure is triggered. This diagnosis immediately highlights both the location (**Line 6**) and the condition leading to the fault ($y < x$),

providing actionable information to the developer. The comprehensible nature of the decision tree makes it an ideal candidate for diagnosis generation. Since the model identifies the faulty line and the conditions leading to failure, developers can systematically target these conditions to design a correct fix.

4.3 Implementation

Our implementation of EFDD is built upon SFLKit [50]. SFLKit provides the means to instrument a Python program under test and collect a trace of execution events. In our implementation, we utilize SFLKit's built-in iteration over the event trace to gather features. Specifically, while SFLKit processes the events to extract various predicates and spectra (such as coverage information), we inject our feature collector into this loop. This feature collector constructs the features based on the extracted data. Once the feature vectors are constructed, we convert them into data frames suitable for feeding directly into a machine-learning classifier. We chose scikit-learn's decision tree learner as the backbone of our diagnosis. Although scikit-learn offers a variety of classifiers, we restricted ourselves to decision trees for several reasons: (a) The inherent explainability of decision trees, as detailed in Section 4.1. (b) The focus of this work is on the overarching approach, not on classifier comparisons.

5 Evaluation

To assess the effectiveness of EFDD, we designed our evaluation around the following key research question:

RQ4: Diagnosis' Quality. How accurate are the diagnoses generated by EFDD from Section 4?
RQ5: Scalability. Does EFDD scale to large programs and complex execution traces?
RQ6: Efficiency. How efficient is EFDD in terms of execution time?

5.1 Experimental Setup

The evaluation focuses on measuring EFDD's capability to generate accurate and interpretable diagnoses by assessing how effectively the generated diagnosis can classify the presence of faults based on program executions. Specifically, we evaluate the diagnosis as a classifier that distinguishes between failing and passing runs, aligning with established practices in this research domain [10, 23]. Unlike prior approaches such as alhazen and Avicenna, which evaluate both the generative and predictive capabilities of their models, our evaluation exclusively focuses on the predictive accuracy of the diagnosis. This distinction stems from the fact that EFDD does not generate new program inputs, a limitation discussed further in Section 7.2, but instead builds diagnoses based on existing execution data.

To ensure a rigorous and meaningful evaluation, we selected subject programs based on the following criteria:

Fault Impact on Output. Each subject must include a fault that affects the program's state and, consequently, its output, ensuring that the fault is observable through test executions and making diagnosis feasible.

Emphasis on Functional Bugs. Most subjects focus on functional bugs that cause incorrect outputs rather than bugs that lead to exceptions or crashes. Functional bugs often require deeper analysis, aligning with EFDD's core strength. While exceptions can signal faults explicitly, functional bugs pose a greater diagnostic challenge.

Realistic and Isolated Defects. Subjects are chosen to reflect real-world defects. Each subject contains exactly one bug to isolate the diagnostic process. Although EFDD is currently designed to analyze one fault at a time, it can handle multiple bugs in a single program if tests clearly distinguish between them, allowing separate diagnoses for each.

Availability of Ground Truth. Each subject includes a known ground truth, enabling a direct comparison between the diagnosed fault and the actual defect, which is critical for quantitatively assessing the accuracy of EFDD's diagnoses.

Labeled Test Cases. Each subject provides at least one set of pre-labeled test cases, including failing and passing tests. This initial labeled set serves as a foundation for training and evaluating EFDD and ensures that faults are triggered and avoided across different test executions.

Based on our benchmark requirements, we selected REFACTORY [16] for our evaluation. REFACTORY is a comprehensive benchmark of student submissions for five distinct programming tasks. It provides initial input-output pairs and a correct reference implementation for each task, making it an ideal candidate for evaluating fault localization and diagnosis tools like EFDD. For each subject in the benchmark, we label the generated inputs as either *passing* or *failing* by comparing the subject's actual output against the expected output from the correct implementation. A test is labeled as passing if the program's output matches the expected output, and it is labeled as failing if the program's output deviates from the expected result. We then use these labeled examples to diagnose each subject based on the initial seed inputs. However, we apply a filter to exclude uninformative subjects that either fail or pass on all the initial seeds.

To assess the accuracy of the diagnoses generated by EFDD, we created 400 labeled evaluation input examples for each subject. We evaluated the model's ability to classify passing and failing runs accurately. Based on the evaluation input, we compute the achieved accuracy, precision, recall, and F1 score. To facilitate an efficient analysis, we evaluate all considered metrics (i.e., accuracy, precision, recall, F1 score, and their macro counterparts) for all subjects not by averaging, but by considering both correctly labeled and mislabeled predictions.

Additionally, we conduct a comprehensive analysis of the interpretability characteristics of the generated decision trees, including their structural complexity (depth, number of leaves), decision structure patterns, and feature utilization patterns. This interpretability analysis assesses whether the generated diagnoses remain comprehensible to developers across different fault scenarios, measuring the complexity distributions of the trees and the practical cognitive load imposed on developers during debugging sessions.

Furthermore, we conduct a case study on real-world subjects to demonstrate the scalability of EFDD. We have selected six subjects from the benchmark that are representative of large and complex systems and support TESTS4PY's built-in test generation. To start, we generate six starting constellations of various numbers of passing and failing tests for each subject and repeat the evaluation for each subject five times to ensure statistical significance. This results in a total of 30 diagnoses per subject, and an overall total of 180 evaluation subjects. We then validate the diagnoses against 200 labeled evaluation inputs, consisting of 100 passing and 100 failing, which we generate for each subject. We evaluate the diagnoses leveraging the same metrics as for REFACTORY.

To assess the efficiency of EFDD, we measure the execution time for each diagnosis generation. With these measurements, we can calculate the overhead of EFDD in terms of the time spent on diagnosis generation, rather than just executing the program.

5.2 RQ4: Diagnosis Quality

For the quality of the diagnosis, we consider the 1777 subjects that pass our requirements from the REFACTORY benchmark. Table 8 comprises all our results over all subjects for each benchmark.

Our approach generated diagnoses that can distinguish between passing and failing executions with an overall accuracy of 89.36% and a macro F1 score of 0.89, demonstrating the predictive power of these diagnoses. On 884 subjects from REFACTORY, we could infer diagnoses that distinguished all runs from our evaluation set, meaning that in 50% of the cases, the diagnoses were as accurate as possible in our evaluation setup. Overall, all our evaluation metrics show stable values without significant outliers, demonstrating again how powerful and balanced, in terms of not favoring non-buggy features over buggy ones since they are more represented in the training sets, the diagnoses generated by EFDD can be. In addition, an average execution time of EFDD of 4.19 seconds on our evaluation machines is a manageable workload with a highly beneficial outcome.

From a small set of given labeled tests, EFDD can generate diagnoses that differentiate between unseen passing and failing program runs with high predictive power, implying the high accuracy of the diagnoses.

Decision Tree Interpretability Analysis. To further demonstrate the interpretability benefits of EFDD, we comprehensively analyzed the decision tree complexity and structure across all 1,777 subjects that were successfully diagnosed. This analysis provides critical insights into the practical interpretability of the generated diagnoses, addressing whether the decision trees remain comprehensible to developers in real-world debugging scenarios.

Table 8. Quality of the generated diagnoses by EFDD.

REFACTORY	BUG	NO-BUG
Precision	0.8891	0.8833
Recall	0.9032	0.8668
F1 Score	0.8961	0.8750
Accuracy		88.65%
macro *Precision*		0.8862
macro *Recall*		0.8850
macro *F1 Score*		0.8855
# of subjects		1777
avg execution time		4.19s
avg loc per subject		14.13

Tree Complexity Distribution. The generated decision trees exhibit elementary structures strongly supporting EFDD's interpretability claims. With a mean depth of 1.15 and a median depth of 1.00, most diagnoses can be understood through a single decision point. Similarly, the mean number of leaves (2.17) and median (2.00) indicate that most diagnoses present developers with only two to three distinct execution paths to consider. This simplicity directly translates to reduced cognitive load during debugging sessions.

We categorized decision trees into four complexity levels based on their structural characteristics: (1) **Simple trees** (n=1261, 71%): Trees with minimal branching requiring basic logical reasoning; (2) **Moderate trees** (n=371, 21%): Trees with moderate complexity but still easily interpretable; (3) **Complex trees** (n=116, 7%): Trees requiring more careful analysis but remaining within manageable bounds; and (4) **Very complex trees** (n=29, 1%): Trees with higher complexity that may challenge interpretation. The overwhelming prevalence of moderate and straightforward trees (92%) demonstrates that EFDD consistently generates interpretable diagnoses across diverse fault scenarios.

Decision Structure Analysis. Further analysis of the decision tree structures reveals that 1261 subjects (71%) resulted in binary decision trees—the most interpretable form where developers face simple yes/no decisions at each node. An additional 362 subjects (20%) involved trees with minimal additional branching, while only 154 subjects (9%) required more complex decision structures. This distribution reinforces EFDD's capability to distill complex execution behaviors into simple, actionable diagnostic rules.

Feature Utilization Patterns. The analysis of feature utilization provides insights into which execution characteristics most effectively drive interpretable diagnoses. With an average of 1.10 features per diagnosis, EFDD achieves high diagnostic accuracy while maintaining simplicity. The most frequently utilized features include scalar pairs (348 cases), variable properties (322 cases), and def-use pairs (255 cases), indicating that data flow relationships are central to interpretable fault diagnosis. Notably, traditional coverage-based features like lines (131 cases) and functions (7 cases) appear less frequently in the final diagnoses, suggesting that EFDD's rich feature set enables more precise and informative diagnostic rules than conventional approaches.

Practical Interpretability Implications. These results demonstrate that EFDD successfully addresses the interpretability challenge in automated debugging. The predominance of simple, shallow decision trees means that developers can quickly understand the diagnostic reasoning without requiring extensive training in machine learning or complex logical analysis. Furthermore, the focus on data flow features aligns with developers' natural debugging intuitions, where understanding variable relationships and value propagation is fundamental to fault comprehension.

EFDD generates highly interpretable diagnoses, with 92% of decision trees classified as simple or moderate complexity, featuring minimal depth (mean: 1.15) and clear decision structures that align with developers' natural debugging processes.

5.3 RQ5: Case Study: Scalability

To assess EFDD's scalability to real-world programs, we conducted a comprehensive case study using six representative subjects from the TESTS4PY dataset. These subjects were selected for their substantial size and complexity, with an average of 1.3K lines of code per subject, representing a significant scale-up from the REFACTORY benchmark used in RQ4. Each subject underwent rigorous evaluation through 30 independent diagnosis generations (6 starting configurations, each repeated 5 times), resulting in 180 evaluations validated against 200 labeled test inputs per subject.

Table 9. Scalability. Results of the scalability evaluation of EFDD on six subjects from TESTS4PY.

TESTS4PY	BUG	NO-BUG
Precision	0.8669	0.9614
Recall	0.9574	0.8784
F1 Score	0.9099	0.9180
Accuracy		91.42%
macro *Precision*		0.9142
macro *Recall*		0.9179
macro *F1 Score*		0.9140
# of configurations		180
avg execution time		14.97s
avg loc per subject		1.3K

Diagnostic Accuracy at Scale. Table 9 demonstrates that EFDD maintains exceptional diagnostic accuracy when scaling to larger, more complex programs. The approach achieves an overall accuracy of 91.42% with a macro F1 score of 0.914, demonstrating minimal degradation compared to the 89.36% accuracy observed on the smaller REFACTORY subjects. The balanced precision (0.9614 for non-buggy, 0.8669 for buggy) and recall (0.8784 for non-buggy, 0.9574 for buggy) metrics indicate robust performance across failure detection and correct execution classification. These results prove that EFDD's diagnostic capabilities remain reliable as program complexity increases.

Scalability of Interpretability. Despite the increased program complexity, the generated decision trees maintain remarkable interpretability characteristics that closely align with the simpler REFACTORY subjects. The mean depth of 1.27 (compared to 1.15 for RQ4) and median depth of 1.00 demonstrate that even for larger programs, EFDD continues to generate shallow, easily interpretable decision structures. Similarly, the mean leaves (2.27) and median leaves (2.00) remain within the range developers can readily comprehend during debugging sessions.

The complexity distribution reinforces these findings: 158 subjects (87.8%) resulted in simple decision trees, 10 subjects (5.6%) produced moderate complexity trees, and only 12 subjects (6.7%) required more complex structures. This distribution closely mirrors the results from RQ4, indicating that EFDD's interpretability benefits scale effectively to real-world program sizes. The predominance of binary decision trees (158 cases, 87.8%) further supports the practical utility of these diagnoses in actual debugging scenarios.

Feature Utilization at Scale. The feature utilization analysis reveals interesting adaptations to larger program contexts while maintaining diagnostic effectiveness. With an average of 1.11 features per diagnosis (nearly identical to RQ4's 1.10), EFDD continues to achieve high accuracy through parsimonious feature selection. Notably, scalar pair features dominate the diagnoses (163 occurrences), reinforcing our earlier finding that data flow relationships are crucial for fault diagnosis across program scales. The relatively lower utilization of traditional coverage features, such as lines (6 cases) and functions executed (not leveraged), suggests that

EFDD's rich feature set enables more precise diagnostic rules, even in complex codebases. Interestingly, string-related features (50 occurrences) show higher prominence in larger programs, potentially reflecting the increased complexity of data processing in real-world applications.

Computational Efficiency. The average execution time of 14.97 seconds per diagnosis represents a reasonable overhead for the diagnostic value provided, especially considering the average of 1.3K lines of code per subject. This timing includes the complete EFDD pipeline: program instrumentation, test execution, feature extraction, and decision tree training. While this represents an increase from the 4.19 seconds observed for smaller REFACTORY subjects, the time complexity scales reasonably with program size, providing actionable diagnostic insights that can significantly reduce overall debugging time.

EFDD successfully scales to real-world program sizes, maintaining high diagnostic accuracy (91.42%) and interpretability (87.8% simple trees, mean depth 1.27) while providing valuable fault diagnoses for complex codebases with reasonable computational overhead.

5.4 RQ6: Efficiency

To assess EFDD's computational efficiency, we measured execution times across both benchmark datasets and compared them against baseline program execution without diagnosis generation.

For the REFACTORY benchmark, EFDD requires an average of 4.19 seconds per diagnosis compared to a 0.45-second baseline (program execution only). However, the average overhead is a factor of 0.55 when comparing individual runs and forming the mean over all subjects. For the larger TESTS4PY subjects, diagnosis generation takes 14.97 seconds on average versus a 4.32-second baseline. The average overhead factor here is 2.4. This overhead encompasses the entire diagnostic pipeline, including program instrumentation, test execution, feature extraction, and decision tree training.

As expected, the overhead increases with program complexity but remains reasonable for practical debugging scenarios. Manual debugging sessions typically span hours or days, so the seconds-to-minutes overhead represents a minimal investment for the diagnostic insights provided.

EFDD introduces reasonable computational overhead (0.55 for smaller programs, 2.40 for larger programs) that scales predictably with program complexity, providing efficient diagnosis generation suitable for practical debugging workflows.

6 Threats to Validity

6.1 Execution Features Study

Internal Validity. One potential threat to internal validity is the accuracy of the event collection process. Any inaccuracies in capturing execution features could lead to incorrect correlations between features and failures. We ensured that the instrumentation and data collection processes were consistent across all experiments to minimize this variation. Another concern is the potential for biases in the test cases provided by the TESTS4PY dataset, which we addressed by verifying that the test cases failed and passed as expected in the buggy and fixed versions.

External Validity. Selecting projects from the TESTS4PY dataset primarily threatens our study's external validity. While this dataset offers a diverse set of Python projects, the results may not generalize to projects written in other programming languages or those with different characteristics, such as size or complexity. Additionally, the dataset's focus on open-source projects may only partially represent the range of failures in proprietary or industrial software.

Construct Validity. Construct validity concerns arise from the metrics used to evaluate correlation and fault localization. To mitigate this risk we included multiple well-established coefficients (TARANTULA, OCHIAI, D*, NAISH2, and GP13) and evaluation metrics (Top-1, Top-5, Top-10, EXAM score, and wasted effort). However, these metrics may only partially align with practical debugging experiences [44]. However, we attempted to address this by providing a comprehensive analysis across multiple metrics and cross-verified our results with these.

Like traditional spectrum-based fault localization (SBFL) techniques [20], our approach shares the fundamental challenge of distinguishing *correlation* from *causation* in statistical debugging. Individual execution features that consistently appear in failing executions might be coincidental rather than causally related to the failure, particularly when test suites exhibit systematic biases. However, EFDD's decision tree approach offers several advantages in addressing this limitation compared to traditional SBFL techniques. By learning *combinations* of execution features rather than analyzing individual features in isolation, decision trees can capture more complex causal relationships that better reflect the actual fault mechanisms. The conjunctive conditions in decision tree paths (e.g., "Line 6 is executed *and* $y < x$") are more likely to represent genuine causal patterns than simple individual feature correlations, as they require multiple conditions to hold for failure prediction simultaneously. Additionally, the interpretable nature of decision trees allows developers to assess the plausibility of the learned conditions, enabling them to distinguish between spurious correlations and meaningful causal relationships through domain knowledge. While our diagnoses should still be understood as identifying *conditions strongly associated with failure* rather than definitive causal proofs, the multi-feature decision tree approach provides a more robust foundation for causal inference than traditional correlation-based fault localization.

Conclusion Validity. Conclusion validity could be impacted by statistical methods correlating execution features with failures. These methods should be revised to prevent incorrect conclusions about the relevance of specific features. We mitigated this threat by employing established statistical fault localization techniques and cross-verifying our results with multiple metrics. Additionally, using a single dataset limits the diversity of our empirical analysis, potentially affecting the robustness of our conclusions. Future work could involve replicating the study on additional datasets to strengthen the generalizability of our findings.

6.2 Execution Features Driven Debugging

Internal Validity. Concerning the implementation of our diagnosis approach, we cannot verify that it accurately follows the exact approach presented in Section 4. However, the results of our experiments, as shown in Table 8, should eliminate the risk of any significant flaws in our implementation. Moreover, the manual inspection of the diagnostic power of our approach in Section 4.2 reinforces this claim. Regarding our experiments for RQ1 we leveraged our implemented test generation for REFACTORY that may be (a) incomplete, i.e., there are possible tests that will not be generated, (b) easy to distinguish by simple features, (c) or produce inputs that the program was not designed to handle. For the test generation of REFACTORY, we aimed to adhere to the tests provided by this benchmark and introduce randomness to cover as much of the input space as possible. Moreover, all generated inputs produced a result when executed on the correct implementation for each question, showing that they are at least accepted. However, when considering the test generation and our implementation, another threat could be that we learn diagnoses based on the execution features, but instead overfit to the generators, i.e., learn only to distinguish passing and failing runs that the generator produces. We tried to eliminate this risk by ensuring that the generators were as general as possible (considering that they should still trigger the fault) and assuming an adequate number of unseen inputs for our evaluation sets.

External Validity. The selection of our subjects might not be sufficient to show the applicability of our approach to an unseen real-world program. This threat might be heavy, especially considering the subjects in REFACTORY student submissions to relatively small tasks. Since the critical part of our approach is the underlying SFLKIT [50]

as its base, our approach should be applicable whenever SFLKit is, which was already verified and tested on the entire BugsInPy [58] benchmark. For this reason, we consider that EFDD applies to actual programs, and our evaluation's results are generalizable and can be transferred to unseen faults.

Since our evaluation is done in Python, we cannot mitigate the risk that EFDD does not apply to other programming languages. However, arguing that our execution features are general and could also be extracted from programs implemented in other programming languages, like C or Java, we are convinced that our technique is general.

Construct Validity. Another concern is the metrics we have chosen to evaluate the results for RQ1, which might be insufficient to demonstrate our diagnosis quality. We leveraged several established metrics (precision, recall, F1 score, and accuracy) to counter this threat for all our measurements.

Conclusion Validity. The statistical methods used to evaluate the quality of the diagnosis could affect the validity of the conclusion. To mitigate this risk, we cross-verified our results with multiple metrics.

7 Related Work

7.1 Fault Localization

Fault localization is a critical area in software debugging. It aims to pinpoint the exact locations in the code responsible for failures. Several traditional techniques have been extensively studied, particularly statistical methods that introduce similarity coefficients such as TARANTULA [19], OCHIAI [2], D* [60], GP [62], AMPLE [7], JACCARD [6], and many others. Spectrum-based fault localization (SBFL) techniques, including those mentioned, are particularly notable for using execution traces to estimate the likelihood that specific code elements are faulty statistically. In recent years, these techniques have gained traction in automated debugging, particularly within automatic program repair frameworks, where they significantly contribute to identifying code regions for repair [37, 46].

Other research in this area has shifted the focus from using lines as the unit for localization to other coverage-based information. Zhang et al. [70] examined basic blocks, Le et al. [29] investigated paths, Jiang et al. [18] leveraged variables and decision trees, and Vancsics et al. [54] considered call frequency. Ribeiro et al. [47] utilized data flow for fault localization. The work by Yu et al. [69] leverages data and control dependency models to calculate suspiciousness scores. Masri [38] proposed fault localization based on information flow coverage, showing that information flows provide a good model for complex interactions between program elements. Moreover, Masri et al. [39] conducted an empirical study identifying factors that reduce the effectiveness of coverage-based fault localization. Koca et al. [25] applied spectrum-based fault localization to concurrency faults by instrumenting programs to create versions with specific thread interleavings. In contrast, Yan et al. [65] employ traditional SBFL but modify the scores afterward based on the context in which the fault propagates. Other approaches do not consider coverage of the traditional SBFL. Soremekun et al. [52] present an approach that builds on program slices for localization, while Papadakis et al. [42, 43] leverage mutation analysis. Abreu et al. [1] explored generic program invariants as error detectors within spectrum-based fault localization, demonstrating that simple bit-mask and range invariants can achieve diagnostic performance similar to traditional approaches. Golagha et al. [12] introduced a comprehensive failure diagnosis toolchain that combines data generation, failure clustering, and fault localization to help developers reduce the time spent on failure analysis.

The extensive research in this area has led to numerous metrics designed to assess the correlation between code locations and failures [9, 26, 41]. Parnin et al. [44] critically evaluate these techniques and question their practical applicability for developers. They conclude that while these approaches can reduce the search space, they often overwhelm developers with numerous potential fault locations, making it challenging to pinpoint the exact faulty lines. Another comprehensive survey by Soremekun et al. [51] investigates general assumptions

of fault localization, finding that most developers would prefer a diagnosis to locations for debugging. Other studies, such as those by Abreu et al. [3], Pearson et al. [45], Heiden et al. [15], and Widyasari et al. [57], have evaluated fault localization techniques across different benchmarks to assess their effectiveness and limitations. In contrast to our work, they did not consider multiple features in their evaluation but concentrated on lines as the unit of measure for localization.

Beyond traditional fault localization, recent research has expanded to leverage these techniques in novel ways. For instance, Le et al. [27] use a learning-to-rank-based approach that integrates knowledge from mined likely invariants [11] to prioritize functions that are more likely to be the cause of faults. Additionally, approaches like ENTBUG [5] utilize fault localization to drive test generation, incorporating localization metrics into the fitness of a genetic test generator.

Considering that statistical fault localization requires executing the entire test suite to provide relevant code locations, Jiang et al. [17] explore how test case prioritization can improve the process. Yoo et al. [68] and Gonzales-Sanchez et al. [13] develop an approach that prioritizes not only execution time reduction but also enhances fault localization results. Gong et al. [8] directly reduce the test suite to achieve similar improvements. In contrast, Xuan et al. [64] purify test cases by splitting them into smaller parts, providing more granular insights, and leveraging these purified tests to improve statistical fault localization. Moreover, test generation can also enhance fault localization results. For example, Artzi et al. [4] demonstrate that generating directed test cases that maximize the similarity between the path constraints of generated tests and those of faulty executions outperforms undirected generation. While our approach seeks to maximize execution differences, the concept aligns with test generation methodologies.

With the rise of machine learning techniques, researchers have explored using various learning techniques for fault localization [31–33, 55, 56, 67]. Notably, feature-based fault localization approaches by Lei et al. [30] abstract program behaviors as features, while Meng et al. [40] leverage semantic features. Lou et al. [36] introduced GRACE, which utilizes graph-based representation learning to fully exploit detailed coverage information by representing coverage as connections between tests and program entities in a graph structure. Yang et al. [66] explored the use of large language models for test-free fault localization, proposing LLMAO, which fine-tunes bidirectional adapter layers on top of LLM representations to localize buggy lines without requiring test coverage information. Additionally, research has investigated the impact of large language models on fault localization [24].

This body of work highlights both the progress made in fault localization and the challenges that remain, particularly in enhancing the practical utility and precision of these techniques for developers.

7.2 Feature-based Debugging

Recent research also uses *features* for deriving debugging diagnoses. ALHAZEN [23] was one of the first approaches that leveraged features for debugging. The approach learns a decision tree from a set of *input features* and iteratively refines the tree by generating new inputs that aim to trigger the failure. From the decision tree, ALHAZEN derives a diagnosis. Similar to ALHAZEN, AVICENNA [10] leverages *input features* for debugging. It follows the same refinement loop but leverages a sophisticated constraint learner to derive a diagnosis and generates new inputs by solving these constraints.

In contrast, our work opens the field for *execution-feature-driven debugging* that can generate diagnoses explaining the actual program behavior that leads to a failure, providing a more comprehensive understanding of the failure, while ALHAZEN and AVICENNA focus on the input space that is not as useful for debugging as the execution space. However, our approach cannot, at this point, effortlessly generate new inputs to trigger the failure, as we cannot directly map the execution features to the input space. Nevertheless, this limitation is mitigated by the fact that ALHAZEN and AVICENNA require a specification of the input space. In contrast, our

approach runs out-of-the-box on any program. We believe that ALHAZEN or AVICENNA would complement our approach.

8 Conclusion and Future Work

First, we conducted an empirical study that explored alternative execution features that could enhance fault localization in software programs. By leveraging the TESTS4PY dataset, we analyzed the correlation between these features and the presence of failures using various statistical fault localization techniques, including TARANTULA, OCHIAI, D*, NAISH2, and GP13. Our findings indicate that incorporating a diverse set of execution features can improve the accuracy and precision of fault localization.

Second, we introduced a novel technique to generate accurate diagnoses that refer to the features investigated in our study, such as lines executed or variable values, making them easy to read and assess.

Our future work will focus on the following topics:

More Programming Languages. To further validate our findings, we will extend the analysis to a broader range of programming languages, building on our initial results.

Test Generation. We aim to map execution features to the input space, allowing us to directly generate new inputs to trigger the fault based on the generated diagnosis.

Automated Repair. In automated repair, the diagnosis inferred by our approach helps to generate patches that fix the fault. For instance, the diagnosis of the `middle()` example from Figure 8 could directly suggest replacing the variable `y` with `x` in the `return` in Line 6 for a program repair.

More Execution Features. Despite our capability to produce precise diagnoses, the performance of our approach depends on its set of execution features. If a failure does not depend on any of the features collected, it will be challenging to produce an accurate diagnosis. This limitation can be countered by adding more features, notably *derived* features such as string or arithmetic properties—but if a failure occurs, say, whenever n is a perfect number, and d is a day with a full moon, it will still be hard to detect these features.

Program Synthesis. By selecting relevant features, we effectively synthesize a *predicate*. Such predicates can also be synthesized through symbolic means [14] and possibly yield better results while still being explainable.

Deep Learning Models. If one can live without explainability, many machine learning models are available that may all result in applicable diagnoses with even higher accuracy. Nevertheless, the decision tree classifier outperformed all other models investigated during our preliminary experiments. However, given the vast number of available models and the frequency with which novel models are introduced, it is worthwhile to investigate this area further, e.g., by conducting a large-scale study.

Testing Oracle. Based on our diagnoses and an adequate mapping approach that enables us to relate features of a program version to an altered version of the same program, we plan to investigate the possibility of generating a testing oracle that can differentiate between passing and failing tests.

User Study. To evaluate the usability of our diagnosis approach, we plan to conduct a user study with developers to assess the effectiveness of the diagnoses generated by our approach.

Data and Tool Availability

Our evaluation data, all scripts, and our EFDD artifact are all open source. The current versions of evaluation scripts and EFDD can be downloaded from

https://github.com/smythi93/efdd

The dataset containing the collected events, features, and all intermediate results is available at

https://doi.org/10.5281/zenodo.14909966

Acknowledgments

This research was partially funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) — ZE 509/7–2 and GR 3634/4–2 Emperor (261444241) and by the European Union (ERC S3, 101093186). Views and opinions expressed are, however, those of the authors only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

References

[1] Rui Abreu, Alberto González, Peter Zoeteweij, and Arjan J. C. van Gemund. 2008. Automatic software fault localization using generic program invariants. In *Proceedings of the 2008 ACM Symposium on Applied Computing* (Fortaleza, Ceara, Brazil) *(SAC '08)*. Association for Computing Machinery, New York, NY, USA, 712–717. doi:10.1145/1363686.1363855

[2] Rui Abreu, Peter Zoeteweij, and Arjan J. C. van Gemund. 2006. An Evaluation of Similarity Coefficients for Software Fault Localization. In *Proceedings of the 12th Pacific Rim International Symposium on Dependable Computing (PRDC '06)*. IEEE Computer Society, USA, 39–46. doi:10.1109/PRDC.2006.18

[3] Rui Abreu, Peter Zoeteweij, Rob Golsteijn, and Arjan J. C. van Gemund. 2009. A Practical Evaluation of Spectrum-based Fault Localization. *J. Syst. Softw.* 82, 11 (Nov. 2009), 1780–1792. doi:10.1016/j.jss.2009.06.035

[4] Shay Artzi, Julian Dolby, Frank Tip, and Marco Pistoia. 2010. Directed test generation for effective fault localization. In *Proceedings of the 19th International Symposium on Software Testing and Analysis* (Trento, Italy) *(ISSTA '10)*. Association for Computing Machinery, New York, NY, USA, 49–60. doi:10.1145/1831708.1831715

[5] José Campos, Rui Abreu, Gordon Fraser, and Marcelo d'Amorim. 2013. Entropy-based Test Generation for Improved Fault Localization. In *Proceedings of the 28th IEEE/ACM International Conference on Automated Software Engineering* (Silicon Valley, CA, USA). Piscataway, NJ, USA, 257–267. doi:10.1109/ASE.2013.6693085

[6] Mike Chen, Emre Kiciman, Eugene Fratkin, Armando Fox, and Eric Brewer. 2002. Pinpoint: problem determination in large, dynamic Internet services. In *Proceedings of the 2002 International Conference on Dependable Systems and Networks*. 595–604. doi:10.1109/DSN.2002.1029005

[7] Valentin Dallmeier, Christian Lindig, and Andreas Zeller. 2005. Lightweight Bug Localization with AMPLE. In *Proceedings of the Sixth International Symposium on Automated Analysis-Driven Debugging* (Monterey, California, USA) *(AADEBUG'05)*. Association for Computing Machinery, New York, NY, USA, 99–104. doi:10.1145/1085130.1085143

[8] Gong Dandan, Wang Tiantian, Su Xiaohong, and Ma Peijun. 2013. A Test-suite Reduction Approach to Improving Fault-localization Effectiveness. *Comput. Lang. Syst. Struct.* 39, 3 (Oct. 2013), 95–108. doi:10.1016/j.cl.2013.04.001

[9] Patrick Daniel and Kwan Yong Sim. 2013. Spectrum-based Fault Localization: A Pair Scoring Approach. *Journal of Industrial and Intelligent Information* 1 (2013), 185–190. doi:10.12720/jiii.1.4.185-190

[10] Martin Eberlein, Marius Smytzek, Dominic Steinhöfel, Lars Grunske, and Andreas Zeller. 2023. Semantic Debugging. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023*, Satish Chandra, Kelly Blincoe, and Paolo Tonella (Eds.). ACM, 438–449. doi:10.1145/3611643.3616296

[11] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. 2007. The Daikon system for dynamic detection of likely invariants. *Sci. Comput. Program.* 69, 1-3 (2007), 35–45. doi:10.1016/j.scico.2007.01.015

[12] Mojdeh Golagha, Abu Mohammed Raisuddin, Lennart Mittag, Dominik Hellhake, and Alexander Pretschner. 2018. Aletheia: a failure diagnosis toolchain. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings* (Gothenburg, Sweden) *(ICSE '18)*. Association for Computing Machinery, New York, NY, USA, 13–16. doi:10.1145/3183440.3183486

[13] Alberto González-Sanchez, Rui Abreu, Hans-Gerhard Gross, and Arjan J. C. van Gemund. 2011. Prioritizing tests for fault localization through ambiguity group reduction. In *26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*, Perry Alexander, Corina S. Pasareanu, and John G. Hosking (Eds.). IEEE Computer Society, 83–92. doi:10.1109/ASE.2011.6100153

[14] Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. 2017. Program Synthesis. *Foundations and Trends in Programming Languages* 4, 1-2 (2017), 1–119. doi:10.1561/2500000010

[15] Simon Heiden, Lars Grunske, Timo Kehrer, Fabian Keller, André van Hoorn, Antonio Filieri, and David Lo. 2019. An evaluation of pure spectrum-based fault localization techniques for large-scale software systems. *Softw. Pract. Exp.* 49, 8 (2019), 1197–1224. doi:10.1002/spe.2703

[16] Yang Hu, Umair Z. Ahmed, Sergey Mechtaev, Ben Leong, and Abhik Roychoudhury. 2019. Re-Factoring Based Program Repair Applied to Programming Assignments. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 388–398. doi:10.1109/ASE.2019.00044

[17] Bo Jiang, Zhenyu Zhang, W. K. Chan, T. H. Tse, and Tsong Yueh Chen. 2012. How Well Does Test Case Prioritization Integrate with Statistical Fault Localization? *Inf. Softw. Technol.* 54, 7 (July 2012), 739–758. doi:10.1016/j.infsof.2012.01.006

[18] Jiajun Jiang, Yumeng Wang, Junjie Chen, Delin Lv, and Mengjiao Liu. 2023. Variable-based Fault Localization via Enhanced Decision Tree. *ACM Trans. Softw. Eng. Methodol.* 33, 2, Article 41 (dec 2023), 32 pages. doi:10.1145/3624741

[19] James A. Jones and Mary Jean Harrold. 2005. Empirical evaluation of the Tarantula automatic fault-localization technique. In *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering* (Long Beach, CA, USA) *(ASE '05)*. Association for Computing Machinery, New York, NY, USA, 273–282. doi:10.1145/1101908.1101949

[20] James A. Jones, Mary Jean Harrold, and John Stasko. 2002. Visualization of Test Information to Assist Fault Localization. In *Proceedings of the 24th International Conference on Software Engineering* (Orlando, Florida). New York, NY, USA, 467–477. doi:10.1145/581339.581397

[21] James A. Jones, Mary Jean Harrold, and John Stasko. 2002. Visualization of Test Information to Assist Fault Localization. In *Proceedings of the 24th International Conference on Software Engineering* (Orlando, Florida). ACM, New York, NY, USA, 467–477. doi:10.1145/581339.581397

[22] René Just, Chris Parnin, Ian Drosos, and Michael D. Ernst. 2018. Comparing developer-provided to user-provided tests for fault localization and automated program repair. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Amsterdam, Netherlands) *(ISSTA 2018)*. Association for Computing Machinery, New York, NY, USA, 287–297. doi:10.1145/3213846.3213870

[23] Alexander Kampmann, Nikolas Havrikov, Ezekiel O. Soremekun, and Andreas Zeller. 2020. When does my program do this? learning circumstances of software behavior. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Virtual Event, USA) *(ESEC/FSE 2020)*. Association for Computing Machinery, New York, NY, USA, 1228–1239. doi:10.1145/3368089.3409687

[24] Sungmin Kang, Gabin An, and Shin Yoo. 2024. A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization. *Proc. ACM Softw. Eng.* 1, FSE, Article 64 (jul 2024), 23 pages. doi:10.1145/3660771

[25] Feyzullah Koca, Hasan Sözer, and Rui Abreu. 2013. Spectrum-Based Fault Localization for Diagnosing Concurrency Faults. In *Testing Software and Systems*, Hüsnü Yenigün, Cemal Yilmaz, and Andreas Ulrich (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 239–254.

[26] David Landsberg, Hana Chockler, Daniel Kroening, and Matt Lewis. 2015. Evaluation of Measures for Statistical Fault Localisation and an Optimising Scheme. In *Fundamental Approaches to Software Engineering*, Alexander Egyed and Ina Schaefer (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 115–129. doi:10.1007/978-3-662-46675-9_8

[27] Tien-Duy B. Le, David Lo, Claire Le Goues, and Lars Grunske. 2016. A learning-to-rank based fault localization approach using likely invariants. In *Proceedings of the 25th International Symposium on Software Testing and Analysis, ISSTA 2016*. 177–188. doi:10.1145/2931037.2931049

[28] Tien-Duy B. Le, David Lo, and Ming Li. 2015. Constrained feature selection for localizing faults. In *Proceedings of the 2015 IEEE International Conference on Software Maintenance and Evolution (ICSME) (ICSME '15)*. IEEE Computer Society, USA, 501–505. doi:10.1109/ICSM.2015.7332502

[29] Wei Le and Mary Lou Soffa. 2010. Path-based fault correlations. In *Proceedings of the Eighteenth ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Santa Fe, New Mexico, USA) *(FSE '10)*. Association for Computing Machinery, New York, NY, USA, 307–316. doi:10.1145/1882291.1882336

[30] Yan Lei, Huan Xie, Tao Zhang, Meng Yan, Zhou Xu, and Chengnian Sun. 2022. Feature-FL: Feature-Based Fault Localization. *IEEE Transactions on Reliability* 71, 1 (2022), 264–283. doi:10.1109/TR.2022.3140453

[31] Xia Li, Wei Li, Yuqun Zhang, and Lingming Zhang. 2019. DeepFL: integrating multiple fault diagnosis dimensions for deep fault localization. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Beijing, China) *(ISSTA 2019)*. Association for Computing Machinery, New York, NY, USA, 169–180. doi:10.1145/3293882.3330574

[32] Yi Li, Shaohua Wang, and Tien N. Nguyen. 2021. Fault Localization with Code Coverage Representation Learning. In *Proceedings of the 43rd International Conference on Software Engineering* (Madrid, Spain) *(ICSE '21)*. IEEE Press, 661–673. doi:10.1109/ICSE43902.2021.00067

[33] Yi Li, Shaohua Wang, and Tien N. Nguyen. 2022. Fault localization to detect co-change fixing locations. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Singapore, Singapore) *(ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 659–671. doi:10.1145/3540250.3549137

[34] Ben Liblit, Mayur Naik, Alice X. Zheng, Alex Aiken, and Michael I. Jordan. 2005. Scalable Statistical Bug Isolation. *SIGPLAN Not.* 40, 6 (jun 2005), 15–26. doi:10.1145/1064978.1065014

[35] Fan Long and Martin Rinard. 2016. An analysis of the search spaces for generate and validate patch generation systems. In *Proceedings of the 38th International Conference on Software Engineering* (Austin, Texas) *(ICSE '16)*. Association for Computing Machinery, New York, NY, USA, 702–713. doi:10.1145/2884781.2884872

[36] Yiling Lou, Qihao Zhu, Jinhao Dong, Xia Li, Zeyu Sun, Dan Hao, Lu Zhang, and Lingming Zhang. 2021. Boosting coverage-based fault localization via graph-based representation learning. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Athens, Greece) *(ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 664–676. doi:10.1145/3468264.3468580

[37] Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. CoCoNuT: combining context-aware neural translation models using ensemble for program repair. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Virtual Event, USA) *(ISSTA 2020)*. Association for Computing Machinery, New York, NY, USA, 101–114. doi:10.1145/3395363.3397369

[38] Wes Masri. 2010. Fault localization based on information flow coverage. *Software Testing, Verification and Reliability* 20, 2 (2010), 121–147. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/stvr.409 doi:10.1002/stvr.409

[39] Wes Masri, Rawad Abou-Assi, Marwa El-Ghali, and Nour Al-Fatairi. 2009. An empirical study of the factors that reduce the effectiveness of coverage-based fault localization. In *Proceedings of the 2nd International Workshop on Defects in Large Software Systems: Held in Conjunction with the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2009)* (Chicago, Illinois) *(DEFECTS '09)*. Association for Computing Machinery, New York, NY, USA, 1–5. doi:10.1145/1555860.1555862

[40] Xiangxin Meng, Xu Wang, Hongyu Zhang, Hailong Sun, and Xudong Liu. 2022. Improving fault localization and program repair with deep semantic features and transferred knowledge. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) *(ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 1169–1180. doi:10.1145/3510003.3510147

[41] Lee Naish, Hua Jie Lee, and Kotagiri Ramamohanarao. 2011. A Model for Spectra-Based Software Diagnosis. *ACM Trans. Softw. Eng. Methodol.* 20, 3, Article 11 (aug 2011), 32 pages. doi:10.1145/2000791.2000795

[42] Mike Papadakis and Yves Le Traon. 2014. Effective fault localization via mutation analysis: a selective mutation approach. In *Proceedings of the 29th Annual ACM Symposium on Applied Computing* (Gyeongju, Republic of Korea) *(SAC '14)*. Association for Computing Machinery, New York, NY, USA, 1293–1300. doi:10.1145/2554850.2554978

[43] Mike Papadakis and Yves Le Traon. 2015. Metallaxis-FL: mutation-based fault localization. *Softw. Test. Verif. Reliab.* 25, 5–7 (aug 2015), 605–628. doi:10.1002/stvr.1509

[44] Chris Parnin and Alessandro Orso. 2011. Are Automated Debugging Techniques Actually Helping Programmers?. In *Proceedings of the 2011 International Symposium on Software Testing and Analysis* (Toronto, Ontario, Canada) *(ISSTA '11)*. Association for Computing Machinery, New York, NY, USA, 199–209. doi:10.1145/2001420.2001445

[45] Spencer Pearson, José Campos, René Just, Gordon Fraser, Rui Abreu, Michael D. Ernst, Deric Pang, and Benjamin Keller. 2017. Evaluating and improving fault localization. In *Proceedings of the 39th International Conference on Software Engineering* (Buenos Aires, Argentina) *(ICSE '17)*. IEEE Press, 609–620. doi:10.1109/ICSE.2017.62

[46] Yuhua Qi, Xiaoguang Mao, Yan Lei, and Chengsong Wang. 2013. Using Automated Program Repair for Evaluating the Effectiveness of Fault Localization Techniques. In *Proceedings of the 2013 International Symposium on Software Testing and Analysis* (Lugano, Switzerland) *(ISSTA 2013)*. Association for Computing Machinery, New York, NY, USA, 191–201. doi:10.1145/2483760.2483785

[47] Henrique L. Ribeiro, P. A. Roberto de Araujo, Marcos L. Chaim, Higor A. de Souza, and Fabio Kon. 2019. Evaluating data-flow coverage in spectrum-based fault localization. In *2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 1–11. doi:10.1109/ESEM.2019.8870182

[48] Raul Santelices, James A. Jones, Yanbing Yu, and Mary Jean Harrold. 2009. Lightweight Fault-Localization Using Multiple Coverage Types. In *Proceedings of the 31st International Conference on Software Engineering (ICSE '09)*. IEEE Computer Society, USA, 56–66. doi:10.1109/ICSE.2009.5070508

[49] Marius Smytzek, Martin Eberlein, Batuhan Serçe, Lars Grunske, and Andreas Zeller. 2024. Tests4Py: A Benchmark for System Testing. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (Porto de Galinhas, Brazil) *(FSE 2024)*. Association for Computing Machinery, New York, NY, USA, 557–561. doi:10.1145/3663529.3663798

[50] Marius Smytzek and Andreas Zeller. 2022. SFLKit: A workbench for statistical fault localization. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Singapore, Singapore) *(ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 1701–1705. doi:10.1145/3540250.3558915

[51] Ezekiel Soremekun, Lukas Kirschner, Marcel Böhme, and Mike Papadakis. 2023. Evaluating the Impact of Experimental Assumptions in Automated Fault Localization. In *Proceedings of the 45th International Conference on Software Engineering* (Melbourne, Victoria, Australia) *(ICSE '23)*. IEEE Press, 159–171. doi:10.1109/ICSE48619.2023.00025

[52] Ezekiel Soremekun, Lukas Kirschner, Marcel Böhme, and Andreas Zeller. 2021. Locating faults with program slicing: an empirical analysis. *Empirical Softw. Engg.* 26, 3 (may 2021), 45 pages. doi:10.1007/s10664-020-09931-7

[53] Friedrich Steimann, Marcus Frenkel, and Rui Abreu. 2013. Threats to the validity and value of empirical assessments of the accuracy of coverage-based fault locators. In *Proceedings of the 2013 International Symposium on Software Testing and Analysis* (Lugano, Switzerland) *(ISSTA 2013)*. Association for Computing Machinery, New York, NY, USA, 314–324. doi:10.1145/2483760.2483767

[54] Béla Vancsics, Ferenc Horváth, Attila Szatmári, and Árpád Beszédes. 2021. Call Frequency-Based Fault Localization. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 365–376. doi:10.1109/SANER50967.2021.00041

[55] Xu Wang, Hongwei Yu, Xiangxin Meng, Hongliang Cao, Hongyu Zhang, Hailong Sun, Xudong Liu, and Chunming Hu. 2024. MTL-TRANSFER: Leveraging Multi-task Learning and Transferred Knowledge for Improving Fault Localization and Program Repair. *ACM Trans. Softw. Eng. Methodol.* 33, 6, Article 148 (jun 2024), 31 pages. doi:10.1145/3654441

[56] Ratnadira Widyasari, Gede Artha Azriadi Prana, Stefanus A. Haryono, Yuan Tian, Hafil Noer Zachiary, and David Lo. 2022. XAI4FL: enhancing spectrum-based fault localization with explainable artificial intelligence. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension* (Virtual Event) *(ICPC '22)*. Association for Computing Machinery, New York, NY, USA, 499–510. doi:10.1145/3524610.3527902

[57] Ratnadira Widyasari, Gede Artha Azriadi Prana, Stefanus Agus Haryono, Shaowei Wang, and David Lo. 2022. Real world projects, real faults: evaluating spectrum based fault localization techniques on Python projects. *Empirical Softw. Engg.* 27, 6 (nov 2022), 50 pages. doi:10.1007/s10664-022-10189-4

[58] Ratnadira Widyasari, Sheng Qin Sim, Camellia Lok, Haodi Qi, Jack Phan, Qijin Tay, Constance Tan, Fiona Wee, Jodie Ethelda Tan, Yuheng Yieh, Brian Goh, Ferdian Thung, Hong Jin Kang, Thong Hoang, David Lo, and Eng Lieh Ouh. 2020. BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Virtual Event, USA) *(ESEC/FSE 2020)*. Association for Computing Machinery, New York, NY, USA, 1556–1560. doi:10.1145/3368089.3417943

[59] Eric Wong, Tingting Wei, Yu Qi, and Lei Zhao. 2008. A Crosstab-based Statistical Method for Effective Fault Localization. In *Proceedings of the 2008 International Conference on Software Testing, Verification, and Validation (ICST '08)*. IEEE Computer Society, USA, 42–51. doi:10.1109/ICST.2008.65

[60] W. Eric Wong, Vidroha Debroy, Yihao Li, and Ruizhi Gao. 2012. Software Fault Localization Using DStar (D*). In *2012 IEEE Sixth International Conference on Software Security and Reliability*. 21–30. doi:10.1109/SERE.2012.12

[61] W. Eric Wong, Yu Qi, Lei Zhao, and Kai-Yuan Cai. 2007. Effective Fault Localization using Code Coverage. In *31st Annual International Computer Software and Applications Conference (COMPSAC 2007)*, Vol. 1. 449–456. doi:10.1109/COMPSAC.2007.109

[62] Xiaoyuan Xie, Fei-Ching Kuo, Tsong Yueh Chen, Shin Yoo, and Mark Harman. 2013. Provably Optimal and Human-Competitive Results in SBSE for Spectrum Based Fault Localisation. In *Proceedings of the 5th International Symposium on Search Based Software Engineering - Volume 8084* (St. Petersburg, Russia) *(SSBSE 2013)*. Springer-Verlag, Berlin, Heidelberg, 224–238. doi:10.1007/978-3-642-39742-4_17

[63] Jifeng Xuan and Martin Monperrus. 2014. Test case purification for improving fault localization. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Hong Kong, China) *(FSE 2014)*. Association for Computing Machinery, New York, NY, USA, 52–63. doi:10.1145/2635868.2635906

[64] Jifeng Xuan and Martin Monperrus. 2014. Test case purification for improving fault localization. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, (FSE-22), Hong Kong, China, November 16 - 22, 2014*, Shing-Chi Cheung, Alessandro Orso, and Margaret-Anne D. Storey (Eds.). ACM, 52–63. doi:10.1145/2635868.2635906

[65] Yue Yan, Shujuan Jiang, Yanmei Zhang, Shenggang Zhang, and Cheng Zhang. 2023. A fault localization approach based on fault propagation context. *Inf. Softw. Technol.* 160, C (aug 2023), 11 pages. doi:10.1016/j.infsof.2023.107245

[66] Aidan Z. H. Yang, Claire Le Goues, Ruben Martins, and Vincent Hellendoorn. 2024. Large Language Models for Test-Free Fault Localization. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 17, 12 pages. doi:10.1145/3597503.3623342

[67] Haoran Yang, Yu Nong, Tao Zhang, Xiapu Luo, and Haipeng Cai. 2024. Learning to Detect and Localize Multilingual Bugs. *Proc. ACM Softw. Eng.* 1, FSE, Article 97 (jul 2024), 24 pages. doi:10.1145/3660804

[68] Shin Yoo, Mark Harman, and David Clark. 2013. Fault Localization Prioritization: Comparing Information-Theoretic and Coverage-Based Approaches. *ACM Trans. Softw. Eng. Methodol.* 22, 3, Article 19 (jul 2013), 29 pages. doi:10.1145/2491509.2491513

[69] Kai Yu, Mengxiang Lin, Qing Gao, Hui Zhang, and Xiangyu Zhang. 2011. Locating faults using multiple spectra-specific models. In *Proceedings of the 2011 ACM Symposium on Applied Computing* (TaiChung, Taiwan) *(SAC '11)*. Association for Computing Machinery, New York, NY, USA, 1404–1410. doi:10.1145/1982185.1982490

[70] Zhenyu Zhang, W. K. Chan, T. H. Tse, Bo Jiang, and Xinming Wang. 2009. Capturing propagation of infected program states. In *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on The Foundations of Software Engineering* (Amsterdam, The Netherlands) *(ESEC/FSE '09)*. Association for Computing Machinery, New York, NY, USA, 43–52. doi:10.1145/1595696.1595705